

KARGO: A Kriging-Based Adaptive-Region Global Optimization Method with a Data-Driven Quantile Region of Interest and Sufficient-Decrease Discipline

S. Onyambu

2026-07-01

Abstract

We propose **KARGO** (Kriging-based Adaptive-Region Global Optimization), a Bayesian-optimization method for expensive moderate-dimensional black-box minimisation. KARGO maintains, at every iteration, a local region of interest (ROI) defined as the intersection of two complementary regions: a *data-driven quantile bounding box* obtained from the per-coordinate range of the top- ρ subset of evaluated points, which adapts the *shape* of the ROI to the empirical distribution of responses; and a *scalar trust-region cap* whose radius σ_k is updated under a sufficient-decrease criterion [Diouane et al., 2023, Conn et al., 2009], which controls the *size* of the ROI. The Expected-Improvement acquisition is maximised inside this ROI; the first inner step of every iteration is reserved for a global EI maximisation on the full domain, guaranteeing that the algorithm cannot lock onto a sub-optimal basin. We benchmark KARGO against EGO [Jones et al., 1998], TREGO [Diouane et al., 2023], and TRIKE [Regis, 2016] on a 14-cell synthetic suite spanning $d \in \{2, \dots, 10\}$, with 20 replications per cell and a budget of 150 infill points each. All four methods share a frozen Latin-hypercube design and a bit-identical initial Gaussian-process fit, eliminating surrogate-fit variance from the comparison. With the default hyperparameters ($c_{\text{force}}=0.1, \rho=0.1, \sigma_0=0.5, \text{reset}=\text{soft}, \text{refresh}=\text{always}$), we compare each baseline against KARGO *separately*. Across the qualifying checkpoints $\text{cp} \in \{50, 100, 150\}$, KARGO is matched-or-beaten by EGO on only 13 of 42 cell-checkpoint pairs, by TREGO on 22, and by TRIKE on 27: KARGO strictly beats EGO on 29/42, TREGO on 21/42, and TRIKE on 15/42, and is the single strongest method on the suite. Varying the five trust-region knobs over a grid of levels, a linear regression and analysis of variance identify $\text{refresh}=\text{always}$ and $\rho=0.1$ as the two dominant levers (main-effect $\eta^2 \approx 0.29$ and 0.26), show that soft reset slightly outperforms hard, and find the sufficient-decrease scale c_{force} statistically inert ($\eta^2 < 10^{-3}$). A linear model predicts the optimal knob settings and places KARGO’s defaults inside the 95% confidence interval of that predicted optimum, so the chosen settings are a statistically probable optimum.

Keywords: Bayesian optimization; Kriging; expected improvement; trust region; quantile-based region of interest; sufficient-decrease forcing; sensitivity analysis; benchmark study.

1 Introduction

Black-box global optimization arises whenever the objective is expensive to evaluate, lacks accessible derivatives, and may be non-convex or multi-modal [Jones, 2001, Conn et al., 2009, Larson et al., 2019]. Modern applications span aerospace design [Forrester et al., 2007], hyperparameter optimization for machine-learning models [Snoek et al., 2012, Feurer and Hutter, 2019], optimization of stochastic simulators [Picheny et al., 2013], and the tuning of metaheuristics that generate space-filling and projection designs in computer experiments [Stokes et al., 2023]. Formally, we seek

$$x^* = \arg \min_{x \in \Omega} f(x), \quad \Omega \subset \mathbb{R}^d,$$

under a tight evaluation budget of tens to a few hundred queries.

Bayesian optimization (BO) is the dominant framework for this regime [Moćkus, 1975, Jones et al., 1998, Brochu et al., 2010, Shahriari et al., 2016, Frazier, 2018, Gramacy, 2020]. BO trains a probabilistic surrogate, most commonly a Gaussian process [Rasmussen and Williams, 2006], and selects evaluations by maximising an acquisition function that trades off exploration and exploitation. The Efficient Global Optimization (EGO) algorithm of Jones et al. [1998] pairs Kriging with the Expected Improvement (EI) criterion and remains the canonical instance.

Two failure modes of vanilla EGO motivate trust-region variants. First, in moderate dimensions the EI surface itself becomes multimodal and exploration tends to dominate exploitation [Bouhlel et al., 2016, Shahriari et al., 2016]. Second, with stationary kernels the surrogate cannot easily concentrate effort near promising regions, so a large fraction of the evaluation budget is spent learning the global landscape [Eriksson et al., 2019, Gramacy and Apley, 2015]. Several influential methods address these failures by restricting the EI search to a region around the incumbent: TRIKE adapts a ball radius by a classical ratio test [Regis, 2016]; TuRBO maintains several local surrogates inside ball-shaped trust regions and arbitrates by a bandit policy [Eriksson et al., 2019]; TREGO alternates a global EGO step with a local step inside a ball whose radius σ_k is updated under a sufficient-decrease forcing rule [Diouane et al., 2023]. All three share the assumption that the local search region is a metric ball with a scalar radius.

We argue in this paper that two qualitatively different regularities of an unknown response surface should drive the local ROI: the *shape* of the level sets and the *size* of the basin currently being refined. A radius-only parametrization couples the two and forces a single scalar to negotiate both. We propose KARGO, an alternative architecture that uses two complementary mechanisms in parallel:

1. A **quantile-based bounding box**, obtained from the per-coordinate range of the top- ρ subset of evaluated points (those with the smallest y -values), recentered on the current incumbent. The box is data-shaped: anisotropic when level sets are diagonally aligned, narrow when the top-quantile points concentrate, wide otherwise.
2. A **scalar trust-region cap**, parametrised by a radius σ_k and updated multiplicatively under a sufficient-decrease criterion on the running minimum. The cap prevents the data-driven box from sampling far from the incumbent during phases when the surrogate is still uncertain.

The ROI used to maximise EI at every iteration is the *intersection* of these two regions. To ensure that the algorithm cannot lock onto a sub-optimal basin and miss a competing one, the first inner step of every iteration is a global EI maximisation on the full domain Ω . The remaining inner steps maximise EI inside the local ROI.

The contributions of this paper are:

1. A precise specification of the KARGO algorithm and its hyperparameter conventions (Section 3), including the two control knobs (**reset** and **refresh**) that govern how aggressively the algorithm forgets a stale ROI, and an analysis of when the two regions that form the ROI are nested versus only partially overlapping.
2. A benchmark on 14 synthetic cells against three strong baselines under a strict *frozen-design* / *frozen-GP* protocol (Section 4) that eliminates Latin-hypercube and GP-fit variance from the comparison. Because KARGO is the method under study, we report *three separate head-to-head comparisons*: EGO vs KARGO, TREGO vs KARGO, and TRIKE vs KARGO (Section 5), rather than collapsing all baselines into a single combined tally.
3. A systematic study of the five hyperparameters (c_{force} , ρ , σ_0 , **reset**, **refresh**) (Section 6),

analysed by a linear regression and an analysis of variance, that identifies refresh=always and $\rho=0.1$ as the two dominant levers, ranks soft reset above hard, and shows c_{force} statistically inert. The fitted model is used to predict the best knob settings and to confirm that KARGO’s defaults lie within the confidence interval of that predicted optimum.

4. A discussion (Section 8) of the function-class structure that makes KARGO win or lose against each baseline, with concrete recommendations for practitioners.

2 Background and Related Work

2.1 Bayesian Optimization Framework

Bayesian optimization formalises the search for $\arg \min_{x \in \Omega} f(x)$ as sequential decision making under a probabilistic surrogate model of f [Moćkus, 1975, Jones et al., 1998, Brochu et al., 2010, Frazier, 2018, Shahriari et al., 2016]. The historical roots trace back to Kushner [1964] (one-dimensional global optimization with Wiener-process priors) and to Moćkus [1975] (the modern framing as Bayesian decision theory applied to expensive black-box problems). The formulation was crystallised in Moćkus [2012].

A BO algorithm maintains a posterior over candidate objectives, $p(f \mid \mathcal{D}_t)$, updated after each evaluation $\mathcal{D}_t = \{(x_i, y_i)\}_{i=1}^t$. From this posterior an acquisition function $\alpha(x; \mathcal{D}_t)$ is constructed to score candidate query points, and the next query is $x_{t+1} \in \arg \max_{x \in \Omega} \alpha(x; \mathcal{D}_t)$. The surrogate is most commonly a Gaussian process [Rasmussen and Williams, 2006], although random forests [Hutter et al., 2011] and Bayesian neural networks have also been employed. The popularity of GP surrogates is driven by their closed-form predictive distributions, calibrated uncertainty estimates, and the smoothness of derived acquisition functions [Rasmussen and Williams, 2006, Santner et al., 2018, Gramacy, 2020].

2.2 Kriging Surrogate Models

Kriging [Krige, 1951, Sacks et al., 1989, Santner et al., 2018] models the unknown response as a realisation of a Gaussian process. Following the conventions of Roustant et al. [2012], we write

$$Y(x) = \mu(x) + Z(x), \quad (1)$$

where $\mu(x)$ is a known or parametric trend and $Z(x)$ is a zero-mean square-integrable Gaussian process with covariance $\psi(x, x') = \text{Cov}(Z(x), Z(x'))$. The covariance kernel ψ controls smoothness, range, and amplitude; a tensor-product form parametrised by characteristic length scales $\theta_l > 0$ and process variance σ^2 is standard [Rasmussen and Williams, 2006, Stein, 1999].

The Mat’ern- ν kernel with $\nu = 5/2$, which produces processes that are twice mean-square differentiable, is the default in most GP software for computer experiments [Roustant et al., 2012, Santner et al., 2018, Gramacy, 2020]. We adopt this default throughout. Hyperparameters (σ^2, θ) are estimated by maximum likelihood [Rasmussen and Williams, 2006, Stein, 1999]; all KARGO GP fits use the DiceKriging R package of Roustant et al. [2012].

2.3 Expected Improvement and Acquisition Functions

Given the GP posterior at iteration t , the canonical acquisition function for noiseless minimisation is the Expected Improvement of Moćkus [1975] and Jones et al. [1998]. Let $y_{\min} = \min_{1 \leq i \leq t} y_i$ and $I(x) = \max\{y_{\min} - Y(x), 0\}$; then

$$\text{EI}(x) = \mathbb{E}[I(x)] = s_t(x) [z \Phi(z) + \phi(z)], \quad z = \frac{y_{\min} - \mu_t(x)}{s_t(x)}, \quad (2)$$

where $\mu_t(x)$, $s_t^2(x)$ are the GP posterior mean and variance at x and Φ, ϕ are the standard normal cdf and pdf [Jones et al., 1998].

Alternative acquisitions include the Lower Confidence Bound $\text{LCB}(x) = \mu_t(x) - \kappa s_t(x)$ of Srinivas et al. [2010]; the Knowledge Gradient of Frazier et al. [2009]; and information-theoretic acquisitions such as Entropy Search [Hennig and Schuler, 2012], Predictive Entropy Search [Hernández-Lobato et al., 2014], and Max-value Entropy Search [Wang and Jegelka, 2017]. The Augmented Expected Improvement of Huang et al. [2006] is a popular adaptation to noisy observations. Shahriari et al. [2016] survey these criteria; Frazier [2018] provides a unified tutorial. KARGO uses EI throughout because it remains the dominant choice in moderate-dimensional noiseless problems [Picheny et al., 2013, Roustant et al., 2012] and is the acquisition used by all three of our comparators.

2.4 The Efficient Global Optimization (EGO) Algorithm

EGO [Jones et al., 1998] composes the Kriging surrogate of Section~2.2 with the EI acquisition of Section~2.3. From an initial space-filling design X_0 , typically a Latin hypercube [McKay et al., 1979, Fang et al., 2005, Morris and Mitchell, 1995], EGO iteratively (i) fits a GP to the current data, (ii) maximises EI over Ω , (iii) evaluates f at the maximiser, and (iv) updates the model. EGO is generally regarded as the reference for global optimization on moderate-dimensional, expensive problems with $d \leq 10$ [Jones, 2001, Picheny et al., 2013, Viana et al., 2014]. Bull [2011] established near-optimal convergence rates of an ε -greedy EI variant; Vazquez and Bect [2010] established density of the sample sequence under reasonable assumptions on f . Beyond $d \approx 10$, EGO performance degrades because the global EI surface becomes multimodal and the GP fit deteriorates [Shahriari et al., 2016, Wang et al., 2016, Bouhlef et al., 2016, Kandasamy et al., 2015].

2.5 Trust-Region and Region-Restricted Bayesian Optimization

A natural way to mitigate EGO’s late-stage exploration is to restrict the EI search to a region near the incumbent. We summarise the methods that established the region-restricted BO paradigm.

2.5.0.1 CG-RBF.

Regis and Shoemaker [2007], building on the radial-basis-function global optimizer of Gutmann [2001], propose the Controlled Gutmann RBF algorithm that alternates between local and global candidate sets sampled around the incumbent. CG-RBF is an early example of the principle that local sampling improves data efficiency in late iterations of expensive optimization [Regis and Shoemaker, 2005].

2.5.0.2 TRIKE.

TRIKE (Trust Region Implementation in Kriging-based optimization with EI) by Regis [2016] adapts the classical trust-region framework to a Kriging-EI surrogate. At each iteration, EI is maximised inside a ball $B(x_k^*, \Delta_k)$ of radius Δ_k , updated by a ratio test on $\eta_k = (f(x_k^*) - f(x_{k+1}))/\text{EI}_k$, expanding Δ_k when η_k exceeds a threshold and contracting it otherwise. TRIKE has no explicit global phase; Regis [2016] pairs it with a restart strategy.

2.5.0.3 TuRBO.

Eriksson et al. [2019] maintain m simultaneous trust regions, each with its own local GP fit, arbitrated by an implicit multi-armed bandit policy. TuRBO was designed for high-dimensional problems ($d \gtrsim 20$) and uses Thompson sampling (rather than EI) inside each region. Because

TuRBO replaces both the acquisition (TS instead of EI) and the trust-region topology (a bandit over K TRs rather than one), it sits outside the EI-based trust-region family this paper studies; we therefore include it in related work but not in the empirical comparison, whose four methods share EI so that observed differences are attributable to ROI geometry alone.

2.5.0.4 TREGO.

Diouane et al. [2023] alternate a standard global EGO step with an optional local step inside a trust region

$$\Omega_k = \{x \in \Omega : d_{\min} \sigma_k \leq \|x - x_k^*\| \leq d_{\max} \sigma_k\},$$

parametrised by a scalar step size σ_k . The local step is taken only if the global step fails a sufficient-decrease condition $f(x_{k+1}^{\text{glb}}) \leq f(x_k^*) - \rho(\sigma_k)$, where ρ is a forcing function with $\rho(\sigma)/\sigma \rightarrow 0$. The step size is updated multiplicatively, $\sigma_{k+1} = \gamma \sigma_k$ on success and $\beta \sigma_k$ on failure, with $\beta \in (0, 1)$ and $\gamma > 1$. The construction borrows from the search/poll convergence theory of Kolda et al. [2003] and Conn et al. [2009], and Diouane et al. [2023] prove that TREGO converges to a Clarke stationary point.

2.5.0.5 Common structure of the four methods.

All four methods assume the local search region is a metric ball, ellipsoid, or annulus around the incumbent, with a scalar radius (or per-coordinate widths in TuRBO) updated through a binary success/failure rule. Recent multi-trust-region variants such as MTRBO [Das et al., 2026] extend the bookkeeping to several balls simultaneously, but retain the radius-based parametrization. KARGO departs from this design by combining a data-shaped quantile region with the scalar radius cap rather than relying on a radius alone.

2.6 Hyperparameter Optimization Context

A common application of moderate-dimensional BO is hyperparameter optimization for machine-learning models and metaheuristics. Random search remains a competitive baseline [Bergstra and Bengio, 2012]. Sequential model-based methods such as SMAC (Sequential Model-based Algorithm Configuration) [Hutter et al., 2011] and Gaussian-process Bayesian optimization (GP-BO) [Snoek et al., 2012] dominate when evaluations are expensive. Bandit and multi-fidelity hybrids such as Hyperband [Li et al., 2017] and BOHB (Bayesian Optimization and HyperBand) [Falkner et al., 2018] exploit early stopping; transfer-learning schemes attack the warm-start problem [Wang et al., 2020]. Feurer and Hutter [2019] provide a recent survey.

3 The KARGO Algorithm

KARGO is composed of five mechanisms: the quantile bounding box, the scalar trust-region cap, the sufficient-decrease forcing function, the global EI step at the top of each iteration, and the EI optimization inside the local ROI. Sections~3.2–3.8 describe each in turn; Algorithm~1 ties them together.

3.1 Notation

Let $X \in \mathbb{R}^{n \times d}$ denote the design matrix and $y \in \mathbb{R}^n$ the responses (lower is better). Let $x^* = X_{\arg \min_i y_i, \cdot}$ be the incumbent best, and let $\sigma_k \in (0, 1]$ denote the trust-region radius at outer iteration k . We write $L_\Omega = u - \ell \in \mathbb{R}^d$ for the side lengths of the domain $\Omega = \prod_{j=1}^d [\ell_j, u_j]$.

3.2 The Quantile Bounding Box

After every iteration KARGO recomputes a data-driven bounding box from the top- ρ subset of evaluated points. Let $\tau \subset \{1, \dots, n\}$ index the lowest $\lceil \rho n \rceil$ entries of the current response vector; X_τ the corresponding rows; and x^* the incumbent. Define per-coordinate elite ranges:

$$L_j = \min_{i \in \tau} X_{ij}, \quad U_j = \max_{i \in \tau} X_{ij}, \quad D_j = \frac{1}{2}(U_j - L_j). \quad (3)$$

Then the quantile box is the axis-aligned box of side $2D$ re-centred on the incumbent and clipped to Ω :

$$B_{\text{quant}}(x^*) = [x^* - D, x^* + D] \cap \Omega. \quad (4)$$

The box is data-shaped: when level sets of f are diagonally aligned the box is anisotropic; when the top- ρ subset concentrates around a single basin the box is narrow; when the top- ρ subset is dispersed the box is wide. Crucially, no per-coordinate scalar needs to be tuned and no kernel-derived length scale is assumed. Re-centring on x^* rather than the elite centroid is deliberate: the centroid is a centroid of *past* evidence, whereas EI should be searched around the currently most promising point.

3.3 The Scalar Trust-Region Cap

KARGO maintains a scalar trust-region radius $\sigma_k \in (0, 1]$ in normalised units of the domain width. Following the TREGO convention of [Diouane et al. \[2023\]](#), σ_k is grown on successful iterations and shrunk on failures:

$$\sigma_{k+1} = \begin{cases} \min\{1, \gamma \sigma_k\} & \text{successful iteration,} \\ \beta \sigma_k & \text{failed iteration,} \end{cases} \quad (5)$$

with $\gamma > 1$ and $0 < \beta < 1$. The trust-region box is then

$$B_{\text{tr}}(x^*, \sigma_k) = [x^* - \sigma_k L_\Omega, x^* + \sigma_k L_\Omega] \cap \Omega, \quad (6)$$

where $L_\Omega = u - \ell$ is the per-coordinate domain width. With this convention $\sigma_k = 1$ covers Ω exactly and $\sigma_k = 0.5$ covers a sub-box of half the domain width. We use an ℓ_∞ box rather than the ℓ_2 ball of TREGO; the difference is cosmetic on the box-constrained problems considered here.

3.4 The Local ROI as Intersection

The local ROI used by the inner EI steps is

$$\Omega'_k = B_{\text{quant}}(x^*) \cap B_{\text{tr}}(x^*, \sigma_k). \quad (7)$$

The intersection has two effects. When the top- ρ subset is concentrated (its bounding box is small), the quantile box dominates and the search refines tightly. When the top- ρ subset is spread (its bounding box covers a large fraction of Ω), the σ_k cap dominates and prevents the search from sampling far from the incumbent. The two components are complementary: the quantile box adapts the shape; the σ_k cap controls the size. If the intersection collapses to a near-zero-volume region we fall back to the σ_k cap alone, so the local EI step always has somewhere to query.

3.4.0.1 Geometry of the intersection: nesting versus partial overlap.

Because both boxes are axis-aligned and re-centred on the same point x^* , the intersection (7) is itself a box centred on x^* , with per-coordinate half-width $h_j = \min(D_j, \sigma_k(u_j - \ell_j))$. Two qualitatively different regimes arise, illustrated in Figure~1. (a) *Containment*. If the elite half-width is smaller than the cap in *every* coordinate, $D_j \leq \sigma_k(u_j - \ell_j)$ for all j , then $B_{\text{quant}} \subseteq B_{\text{tr}}$ and the intersection equals B_{quant} exactly: the data-driven box sits entirely inside the trust-region cap, and the cap is inactive (panel a). This is the regime the trust cap is designed for: it simply bounds how far the quantile box may reach while the box itself decides the shape. (b) *Partial overlap*. If instead the quantile box is narrower than the cap in some coordinate but *wider* in another: $D_1 < \sigma_k(u_1 - \ell_1)$ yet $D_2 > \sigma_k(u_2 - \ell_2)$, as happens when the elite points line up along a diagonal or valley so that B_{quant} is strongly anisotropic while B_{tr} is isotropic, then *neither region contains the other*. The cap protrudes beyond the quantile box along x_1 ; the quantile box protrudes beyond the cap along x_2 ; and the intersection Ω'_k is strictly smaller than both (panel b). In this regime the two mechanisms act genuinely jointly: the cap trims the coordinate in which the elite set is over-dispersed, while the quantile box trims the coordinate in which the elite set has already concentrated. Partial overlap is therefore not a degenerate case but the typical situation on anisotropic landscapes, and it is precisely where the data-driven box and the scalar cap each contribute information the other lacks.

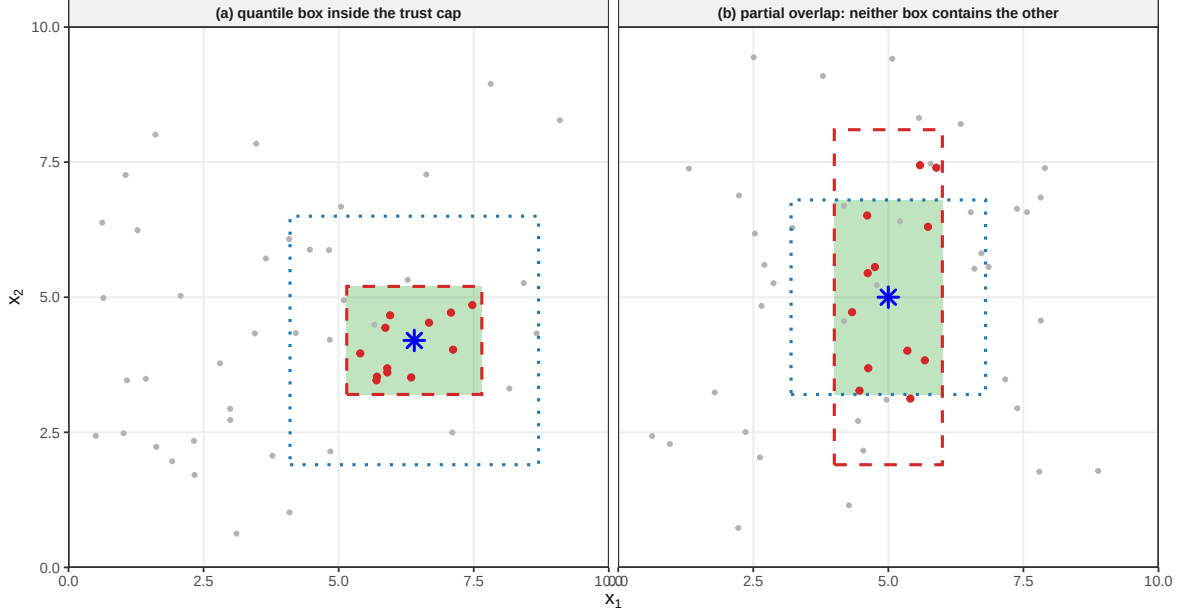


Figure 1: The KARGO local region of interest in two regimes. Grey points are the full design X ; red points are the elite subset ($\lceil \rho n \rceil$ smallest by y); the blue star is the incumbent x^* . The dashed red box is B_{quant} (the elite axis extent (3) re-centred on x^*); the dotted blue box is the isotropic cap B_{tr} (6); the shaded green region is the ROI Ω'_k (7). (a) the quantile box is narrower than the cap in both coordinates, so $B_{\text{quant}} \subseteq B_{\text{tr}}$ and the ROI equals B_{quant} . (b) the quantile box is anisotropic (narrow in x_1 , wide in x_2) so neither box contains the other and the ROI is strictly smaller than both.

3.5 The Sufficient-Decrease Forcing Function

An iteration is declared *successful* if the running-minimum improvement after the iteration's inner steps exceeds a forcing-function threshold:

$$\text{successful at } k \iff f_{\text{before}} - f_{\text{after}} > \rho_{\text{force}}(\sigma_k), \quad (8)$$

with

$$\rho_{\text{force}}(\sigma) = c_{\text{force}} \sigma^2, \quad c_{\text{force}} > 0. \quad (9)$$

The TREGO default in the noise-free deterministic setting is $\rho(\sigma) = \sigma^2$ (i.e. $c_{\text{force}} = 1$); pilots on Hartmann6 with that default showed that real, useful improvements of order 0.05 were filtered out as “insufficient” because $\sigma_k^2 = 0.25$ at the initial $\sigma_0 = 0.5$. A softer c_{force} preserves the asymptotic property $\rho_{\text{force}}(\sigma)/\sigma \rightarrow 0$ that drives the search/poll convergence proof of Kolda et al. [2003], while keeping the threshold proportional to the typical improvement scale observed in BO settings. When an iteration fails the test, σ_{k+1} shrinks and a soft expansion widens Ω'_k by $1/\beta$ around its centre, clipped to Ω . After `counter_max` consecutive failures or whenever σ_k drops below $\sigma_{\min}$, a hard reset returns σ to σ_0 and Ω'_k to Ω .

3.6 Reset and Refresh Policies

Two policies govern the long-run behaviour of the ROI:

reset $\in \{\text{soft}, \text{hard}\}$. Under **soft**, after a failure we widen the current ROI by a factor $1/\beta$ about its centre, floor σ at $\sigma_{\min}$, and never clear state. Under **hard**, after `counter_max` consecutive failures (or once $\sigma < \sigma_{\min}$), we reset σ to σ_0 and the ROI to Ω in full. **hard** forgets accumulated trust-radius information but ensures the algorithm revisits the global domain if a basin proves uncompetitive.

refresh $\in \{\text{lazy}, \text{always}\}$. Under **lazy**, B_{quant} is recomputed only after a successful iteration. Under **always**, B_{quant} is recomputed at the end of every iteration. Section 6 shows **always** dominates **lazy**, because the elite set must be reflected back into the ROI promptly after a reset.

3.7 The Global Step at the Top of Every Iteration

Each iteration has m_{inner} inner EI steps: step $s = 1$ is a global EI maximisation on the full domain Ω ; steps $s = 2, \dots, m_{\text{inner}}$ are local EI maximisations on the current ROI Ω'_k . The unconditional global step is the structural advantage of the design: every iteration includes an exploration query not restricted to the local region. Without it, the local ROI can lock onto a sub-optimal basin; in pilot studies a single failing run on a multimodal landscape (Rastrigin) was enough to make this lock-in visible.

3.8 EI Maximisation Inside a Box

EI is multimodal in moderate dimensions [Jones et al., 1998, Frazier, 2018], particularly inside trust regions that have been recently contracted. KARGO maximises EI inside a box by the `DiceOptim::max_EI` routine of Roustant et al. [2012], which couples a derivative-free evolutionary solver (`genoud`) with closed-form gradient refinement and is robust to the multimodality of EI inside contracted boxes.

3.9 Algorithm

The full KARGO loop is given in Algorithm~1. After all m_{inner} inner steps the sufficient-decrease test (8) fires; the radius updates; and the ROI is rebuilt (always after a success; under **refresh=always**, after a failure too). All GP updates are incremental (`DiceKriging::update`) with covariance and trend re-estimation enabled but no nugget re-estimation.

3.10 Numerical robustness: self-healing on a singular surrogate

As the trust region contracts onto a sharply peaked optimum, the inner EI maximiser proposes points inside an ever-smaller neighbourhood of the incumbent. The resulting near-duplicate, highly correlated design points make the Gaussian-process covariance matrix increasingly ill-conditioned, and the incremental Cholesky factorisation of `DiceKriging::update` eventually

Algorithm 1 KARGO: Kriging-based Adaptive-Region Global Optimization

Require: Initial design X_0 , responses y_0 , GP $\hat{f}_0$, bounds ℓ, u , budget N , parameters $(\sigma_0, \gamma, \beta, \sigma_{\min}, c_{\text{force}}, \rho, m_{\text{inner}}, \text{reset}, \text{refresh}, \text{counter_max})$.

- 1: $X \leftarrow X_0, y \leftarrow y_0, \hat{f} \leftarrow \hat{f}_0, \sigma \leftarrow \sigma_0, \text{fail_cnt} \leftarrow 0, \text{current} \leftarrow \Omega, n \leftarrow |y|$.
- 2: **for** $k = 1, 2, \dots$ **do**
- 3: $f_{\text{before}} \leftarrow \min y$.
- 4: **for** $s = 1$ to m_{inner} **do**
- 5: **if** $|y| - |y_0| \geq N$ **then return** (X, y) .
- 6: **end if**
- 7: $\text{box} \leftarrow \Omega$ if $s = 1$ else current .
- 8: $x_{\text{new}} \leftarrow \arg \max_{x \in \text{box}} \text{EI}_{\hat{f}}(x)$.
- 9: $y_{\text{new}} \leftarrow f(x_{\text{new}})$; append to X, y ; $\hat{f} \leftarrow \hat{f} | (x_{\text{new}}, y_{\text{new}})$.
- 10: **end for**
- 11: $\Delta \leftarrow f_{\text{before}} - \min y$; $x^* \leftarrow X_{\arg \min y, \cdot}$; $\text{succ} \leftarrow [\Delta > c_{\text{force}} \sigma^2]$.
- 12: **if succ then**
- 13: $\sigma \leftarrow \min(1, \gamma \sigma)$; $\text{fail_cnt} \leftarrow 0$; $\text{current} \leftarrow B_{\text{quant}}(x^*) \cap B_{\text{tr}}(x^*, \sigma)$.
- 14: **else**
- 15: $\sigma \leftarrow \beta \sigma$; $\text{fail_cnt} \leftarrow \text{fail_cnt} + 1$.
- 16: **if** $\text{reset} = \text{hard}$ and $(\sigma < \sigma_{\min} \text{ or } \text{fail_cnt} \geq \text{counter_max})$ **then**
- 17: $\sigma \leftarrow \sigma_0$; $\text{current} \leftarrow \Omega$; $\text{fail_cnt} \leftarrow 0$.
- 18: **else**
- 19: expand current by $1/\beta$ about its centre, clipped to Ω ; $\sigma \leftarrow \max(\sigma, \sigma_{\min})$.
- 20: **end if**
- 21: **if** $\text{refresh} = \text{always}$ **then**
- 22: $\text{current} \leftarrow B_{\text{quant}}(x^*) \cap B_{\text{tr}}(x^*, \sigma)$.
- 23: **end if**
- 24: **end if**
- 25: **end for**

fails outright with a “leading minor of order k is not positive” error, a genuine numerical breakdown, not a modelling choice. On most functions this never happens, but on tightly basined cells such as `beale_d02` it can kill a replicate mid-run.

KARGO handles this by *self-healing* rather than by globally perturbing the model. By default a near-duplicate guard is disabled, so every healthy run is bit-for-bit identical to the canonical algorithm above. The first time an update raises the singular-matrix error, the run is restarted from the pristine initial model with the guard armed from the very first infill: any EI proposal lying within a small tolerance (we use 0.5% of the domain width per coordinate, in the ℓ_∞ sense) of an existing design point is rejected and the inner step is retried, so the singular pile-up never forms.

The reason the guard must be armed *from step 1*, rather than switched on at the moment of failure, is that by the time the covariance is singular the offending cluster of points is already in the design and cannot be removed by an incremental update; only preventing the cluster from forming in the first place restores conditioning. The restart therefore replays the whole run under the guard. It fires at most once per replicate (the retry already carries the guard, so any subsequent singularity is treated as genuine and re-raised), it changes only runs that would otherwise have crashed, turning a dead replicate into a completed one, and it leaves every non-failing run untouched. This is what lets the study report a full 20 replications on *every* cell, including `beale_d02`, without resorting to a per-cell nugget inflation or a hand-tuned jitter that would alter the surrogate everywhere.

3.11 Computational Complexity

Per iteration KARGO performs m_{inner} GP updates, each $O(t^3)$ (Cholesky; `DiceKriging::update` uses incremental factorisation); one EI maximisation per inner step, dominated by `genoud` and the same cost as in EGO or TREGO; an $O(t \log t)$ sort to build the quantile box; and an $O(d)$ sigma update. Total per-iteration cost is $O(m_{\text{inner}} t^3) = O(t^3)$, matching EGO; KARGO’s extra book-keeping is asymptotically negligible.

4 Experimental Setup

4.1 Test Functions

We use the 14-cell subset of the Surjanovic–Bingham VLSE library [Surjanovic and Bingham] reported in Table~1. Cells span $d \in \{2, 3, 4, 6, 8, 10\}$ and cover unimodal (Rosenbrock), valley-shaped (Dixon–Price), multimodal (Ackley, Rastrigin, Michalewicz, Shubert) and near-flat (Easom) landscapes. Two cells were excluded outright. `goldpr_d02` (Goldstein–Price) has dynamic range $y \in [98, 2.9 \times 10^5]$, on which the `DiceKriging` maximum-likelihood estimator becomes unbounded slow; `levy_d10` at $d=10$, $N=150$ is intractably slow (pilot runs returned 0 of 80 completed reps after over three hours). The remaining 14 cells discriminate the four methods across all interesting function-class regimes.

cell	d	type	fmin
beale_d02	2	multimodal	0
branin_d02	2	multimodal	0.4
camel6_d02	2	multimodal	-1
easom_d02	2	near-flat	-1
shubert_d02	2	multimodal	-1.9e+02
hart3_d03	3	multimodal	-3.9
colville_d04	4	valley	0
hart6_d06	6	multimodal	-3.3
ackley_d08	8	multimodal	0
ackley_d10	10	multimodal	0
dixonpr_d10	10	valley	0
michal_d10	10	multimodal	-9.7
rastr_d10	10	multimodal	0
rosen_d10	10	unimodal valley	0

Table 1: Test cells used in the benchmark, with known global minimum f_{min} . Cells are referred to throughout by the key in the first column.

4.2 Initial Design and Frozen Gaussian Process

Each cell uses $n_{\text{init}} = 10d$ Latin-hypercube points [McKay et al., 1979, Santner et al., 2018] drawn with seed $1000 + r$ at replication $r \in \{1, \dots, 20\}$. A `DiceKriging::km` model [Roustant et al., 2012] with `Mat{e}rn-5/2` kernel, constant trend, and re-estimated nugget is fit once per (cell, rep) and serialised. *The exact same fitted GP object is loaded by all four methods at the start of their run*, so the comparison is free of GP-fit variance: any differences arise only from the acquisition trajectory.

4.3 Methods Compared

EGO. Standard EI maximisation over Ω via `DiceOptim::fastEGO.nsteps` [Jones et al., 1998, Roustant et al., 2012].

TREGO. Alternating global/local EI with a scalar trust region [Diouane et al., 2023], via `DiceOptim::TREGO.nsteps`.

TRIKE. The TRIKE variant of Regis [2016], invoked as `TREGO.nsteps(..., trcontrol=list(algo="TRIKE"))`

KARGO. Algorithm 1 with the default hyperparameters $c_{\text{force}}=0.1$, $\rho=0.1$, $\sigma_0=0.5$, `reset=soft`, `refresh=always`, $\gamma=1/0.9$, $\beta=0.9$, $\sigma_{\min}=0.02$, `counter_max=3`, $m_{\text{inner}}=5$. The analysis of Section 6 studies the effect of these knobs and confirms the defaults are statistically well-supported.

4.4 Budget, Replications, Checkpoints

For every (method, cell, rep) triple we add $N = 150$ evaluations on top of the initial $10d$. We report the running minimum at four *checkpoints* $\text{cp} \in \{20, 50, 100, 150\}$, where cp is the number of post-initial BO evaluations. Each (method, cell) combination is replicated 20 times with the seeds described above.

4.5 The At-Par-Or-Better Tally

For each ordered pair of methods (A, B) we declare A *at-par-or-better* (APB) than B on a (cell, cp) pair iff its mean running minimum satisfies $\bar{y}_A^* \leq \bar{y}_B^* + \tau$, with tolerance $\tau = \max(10^{-3}, 0.01 |\bar{y}_B^*|)$, i.e. A either strictly improves on B or ties it to within 1%. A *strict win* requires $\bar{y}_A^* \leq \bar{y}_B^* - \tau$. Because KARGO is the method under study, Section~5 reports the three head-to-head comparisons separately. The *sensitivity* study of Section~6 keeps an external TREGO yardstick for scoring each knob setting, so the analysis of variance there isolates the effect of the knobs against a reference that does not move with the setting being scored.

5 Results

We present the benchmark as three separate head-to-head comparisons: EGO vs KARGO (Section~5.2), TREGO vs KARGO (Section~5.3), and TRIKE vs KARGO (Section~5.4), rather than collapsing the baselines into one combined table. Section~5.1 first gives the joint picture, and each comparison then reports a per-checkpoint win/tie/loss tally and the per-cell means at $\text{cp} = 100$. The head-to-head paired Wilcoxon signed-rank tests [Wilcoxon, 1945] at the convergence-tolerance gate, with Bonferroni correction across the 14 cells ($\alpha = 0.05/14 \approx 3.57 \times 10^{-3}$), are collected in Appendix~A.

5.1 Overview

Table~2 reports, for each checkpoint, the number of cells (out of 14) on which each baseline *reaches* KARGO under the 1% tolerance ($\bar{y}_M^* \leq \bar{y}_{\text{kargo}}^* + \tau$); **lower is better for KARGO**, and $14 - \text{count}$ is the number of cells KARGO holds outright. Summing over the qualifying checkpoints $\text{cp} \in \{50, 100, 150\}$, EGO reaches KARGO on only 13 of 42 cell-checkpoint pairs, TREGO on 22, and TRIKE on 27; conversely KARGO strictly beats EGO on 29/42, TREGO on 21/42, and TRIKE on 15/42. Aggregated over the qualifying checkpoints KARGO is the strongest method on the suite, but the ranking is budget- and landscape-dependent. TRIKE is its closest competitor: on the high-dimensional ($d=10$) narrow-valley and multimodal cells TRIKE’s offset ratio-test ball is actually ahead at mid-budget (at $\text{cp} = 100$ it gives the best of the four methods on four of the six $d \geq 8$ cells), and only by the full budget $\text{cp} = 150$ does

KARGO overtake it on most, leading on four of those six, with TRIKE remaining decisively better on the ill-conditioned Dixon-Price valley. EGO is dominated throughout. This budget- and landscape-dependence is precisely why we report the three baselines as separate head-to-head comparisons rather than as one aggregate, and why the discussion (Section~8) is explicit about where KARGO loses. Figure~2 shows the full set of per-cell convergence curves for all four methods.

cp	EGO	TREGO	TRIKE
20	5	9	12
50	4	8	11
100	5	7	9
150	4	7	7

Table 2: Number of cells (out of 14) on which each baseline reaches KARGO (at-par-or-better, tolerance $\max(10^{-3}, 0.01|\bar{y}_{\text{kargo}}|)$), by checkpoint. cp = 20 is reported but not part of the qualifying aggregate. **Lower is better for KARGO.**

5.2 EGO versus KARGO

Against EGO, KARGO is at-par-or-better on all 14 of the 14 cells at every checkpoint, and strictly wins on 9 of them at cp = 100; EGO never strictly beats KARGO on any cell. The convergence curves (the EGO and KARGO series of Figure~2) make the gap visible: EGO disperses its queries across Ω and never concentrates effort, so it trails on the Ackley and Michalewicz cells and merely matches KARGO on the low-dimensional cells that both solve to the global optimum. Table~3 gives the per-checkpoint tally and Table~4 the per-cell means at cp = 100.

cp	KARGO strict win	Tie	KARGO strict lose
20	9	5	0
50	10	4	0
100	9	5	0
150	10	4	0

Table 3: EGO vs KARGO, per checkpoint (out of 14 cells): cells where KARGO strictly wins, ties within tolerance, and strictly loses.

5.3 TREGO versus KARGO

TREGO is the stronger of the two ball-based baselines. KARGO is at-par-or-better than TREGO on 11–12 of the 14 cells across checkpoints, rising to 12 by cp = 150 as KARGO’s elite-box refinement compounds; TREGO strictly wins on only 3 cells: the multimodal-without-dominant-basin plateaus where its annulus step drifts across basins (Section~8.2). The TREGO and KARGO series of Figure~2 show the head-to-head curves; Tables~5–6 give the tally and the cp = 100 means.

5.4 TRIKE versus KARGO

TRIKE is KARGO’s closest competitor early in the budget: its ratio-test ball is aggressive and reaches KARGO on 12 of the 14 cells at cp = 20, where KARGO is at-par-or-better on only 6. As the budget grows the picture reverses: KARGO is at-par-or-better on 11 of 14 by cp = 150, and TRIKE retains strict wins only on the narrow-valley and multimodal $d=10$

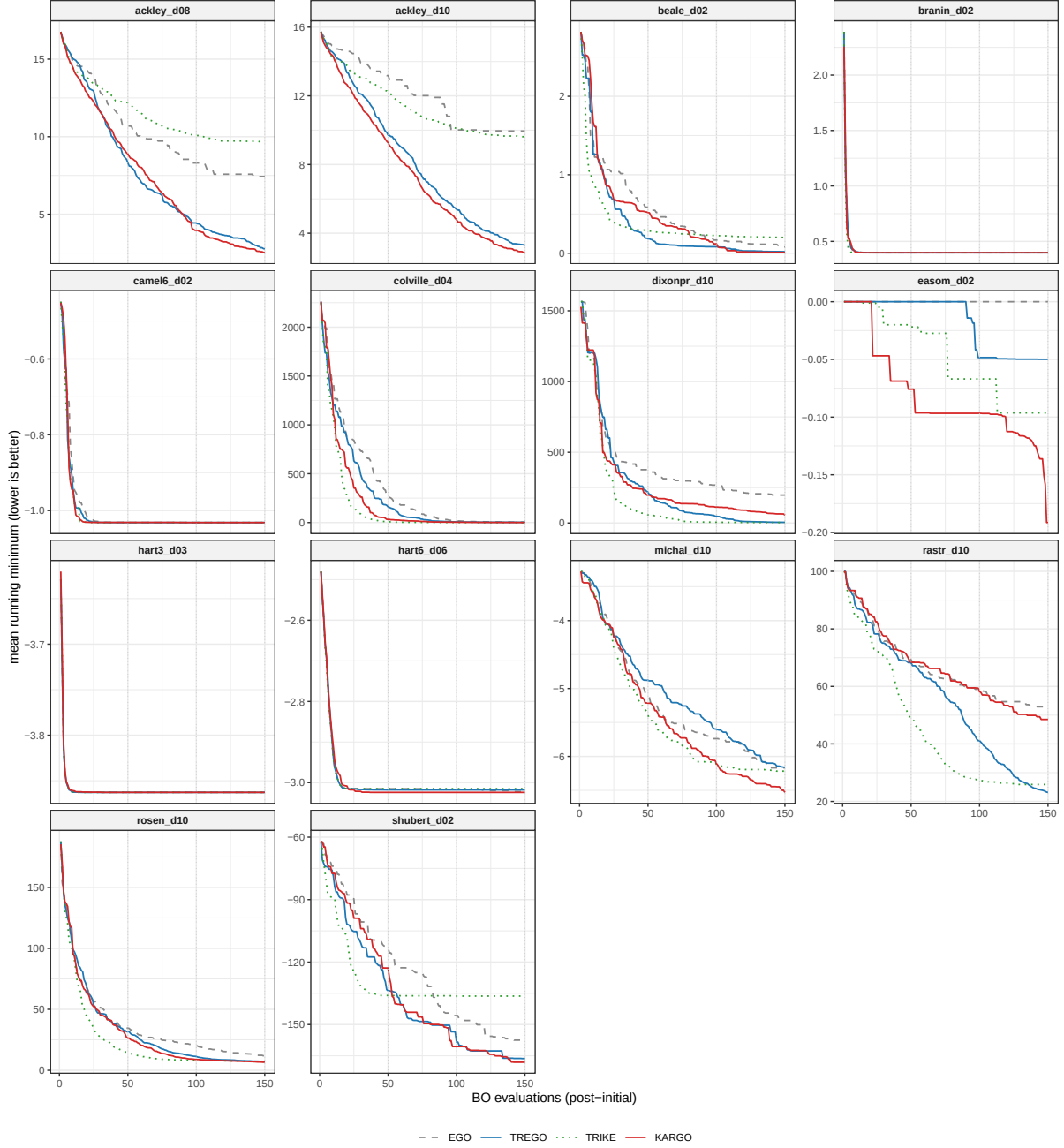


Figure 2: Mean running minimum (over 20 replications) versus number of BO evaluations, per cell, for all four methods. Lower is better.

cells (`dixonpr_d10`, `rastr_d10`, `rosen_d10`) where its offset ball captures gradient-direction information. The TRIKE and KARGO series of Figure~2 and Tables~7-8 give the details.

6 Sensitivity Analysis

To understand which of the five trust-region knobs (c_{force} , ρ , σ_0 , `reset`, `refresh`) drive performance, we evaluate every combination of the levels

$$c_{\text{force}} \in \{0.01, 0.1, 0.5, 0.7, 1, 2\}, \quad \rho \in \{0.1, 0.2, 0.3, 0.5, 0.7\}, \quad \sigma_0 \in \{0.2, 0.5, 0.8, 1\},$$

together with `reset` $\in \{\text{hard}, \text{soft}\}$ and `refresh` $\in \{\text{lazy}, \text{always}\}$. Each combination is run on the full 14-cell suite with 20 replications and scored by its at-par-or-better sum against the external

cell	$f_{\min}$	EGO	KARGO	winner
branin_d02	0.3979	0.3979	0.3979	tie
camel6_d02	-1.032	-1.032	-1.032	tie
easom_d02	-1	-5.418e-07	-0.09672	KARGO
shubert_d02	-186.7	-145.7	-160.6	KARGO
beale_d02	0	0.1679	0.1195	KARGO
hart3_d03	-3.863	-3.863	-3.863	tie
colville_d04	0	19.45	3.45	KARGO
hart6_d06	-3.322	-3.017	-3.024	tie
ackley_d08	0	8.307	3.962	KARGO
ackley_d10	0	10.02	4.791	KARGO
rastr_d10	0	58.68	58.14	tie
rosen_d10	0	20.31	8.851	KARGO
dixonpr_d10	0	264.7	112.6	KARGO
micah_d10	-9.66	-5.738	-6.107	KARGO

Table 4: EGO vs KARGO: mean running minimum at $cp = 100$ (lower is better). **Bold** marks the within-tolerance winner.

cp	KARGO strict win	Tie	KARGO strict lose
20	5	6	3
50	6	5	3
100	7	4	3
150	8	4	2

Table 5: TREGO vs KARGO, per checkpoint (out of 14 cells): KARGO strict wins, ties within tolerance, and strict losses.

cell	$f_{\min}$	TREGO	KARGO	winner
branin_d02	0.3979	0.3979	0.3979	tie
camel6_d02	-1.032	-1.032	-1.032	tie
easom_d02	-1	-0.04839	-0.09672	KARGO
shubert_d02	-186.7	-158.4	-160.6	KARGO
beale_d02	0	0.08084	0.1195	TREGO
hart3_d03	-3.863	-3.863	-3.863	tie
colville_d04	0	6.866	3.45	KARGO
hart6_d06	-3.322	-3.018	-3.024	tie
ackley_d08	0	4.435	3.962	KARGO
ackley_d10	0	5.381	4.791	KARGO
rastr_d10	0	41.12	58.14	TREGO
rosen_d10	0	11.31	8.851	KARGO
dixonpr_d10	0	47.65	112.6	TREGO
micah_d10	-9.66	-5.599	-6.107	KARGO

Table 6: TREGO vs KARGO: mean running minimum at $cp = 100$ (lower is better). **Bold** marks the within-tolerance winner.

TREGO baseline,

$$\text{sum_apb} = \sum_{cp \in \{50, 100, 150\}} \#\{\text{cells where the setting is APB vs TREGO}\}, \quad \text{higher is better, maximum 42.}$$

cp	KARGO strict win	Tie	KARGO strict lose
20	2	4	8
50	3	4	7
100	5	5	4
150	7	4	3

Table 7: TRIKE vs KARGO, per checkpoint (out of 14 cells): KARGO strict wins, ties within tolerance, and strict losses.

cell	$f_{\min}$	TRIKE	KARGO	winner
branin_d02	0.3979	0.3982	0.3979	tie
camel6_d02	-1.032	-1.032	-1.032	tie
easom_d02	-1	-0.06691	-0.09672	KARGO
shubert_d02	-186.7	-136.3	-160.6	KARGO
beale_d02	0	0.2237	0.1195	KARGO
hart3_d03	-3.863	-3.863	-3.863	tie
colville_d04	0	1.18	3.45	TRIKE
hart6_d06	-3.322	-3.015	-3.024	tie
ackley_d08	0	10.09	3.962	KARGO
ackley_d10	0	10.1	4.791	KARGO
rastr_d10	0	27.36	58.14	TRIKE
rosen_d10	0	8.185	8.851	TRIKE
dixonpr_d10	0	5.162	112.6	TRIKE
michal_d10	-9.66	-6.117	-6.107	tie

Table 8: TRIKE vs KARGO: mean running minimum at cp = 100 (lower is better). **Bold** marks the within-tolerance winner.

We use these runs for two purposes: to identify, by a linear regression and an analysis of variance, which knobs actually drive `sum_apb` (Section~6.1); and to predict the best knob settings and test whether KARGO’s defaults are a statistically probable optimum (Section~6.2).

6.1 Linear Regression and Analysis of Variance

Because c_{force} , ρ and σ_0 are *ordinal* numeric levels rather than unordered labels, we code them with orthogonal polynomial contrasts (linear `.L'`, quadratic `.Q'`, cubic `.C'`, and so on) and fit the full-factorial model with all main effects and two-factor interactions,

$$\text{sum_apb} \sim (\text{ord}(c_{\text{force}}) + \text{ord}(\rho) + \text{ord}(\sigma_0) + \text{reset} + \text{refresh})^2.$$

This model explains $R^2 = 0.82$ of the variance (adjusted 0.78). The polynomial coding leaves the by-term sum-of-squares unchanged but lets us read curvature straight from the coefficients. Table~9 is the analysis of variance: **refresh** and ρ dominate, followed by σ_0 and **reset**, with significant $\rho \times \text{refresh}$, $\sigma_0 \times \text{refresh}$, $\rho \times \sigma_0$, **reset** $\times$ **refresh** and $\sigma_0 \times \text{reset}$ interactions. c_{force} **is inert**: its main effect and all four of its interactions are non-significant (each $p > 0.48$).

6.1.0.1 Significant curvature.

The coefficient summary shows two clear *quadratic* effects. For ρ the linear term is large and negative ($\hat{\beta}_{\rho,L} = -5.52$) while the quadratic term is positive and significant ($\hat{\beta}_{\rho,Q} = 2.01$, $p < 10^{-3}$): performance falls steeply as ρ leaves its smallest level and then flattens, a convex, diminishing-harm shape that pins the optimum at $\rho = 0.1$. The same pattern recurs, more mildly,

Term	Df	Sum Sq	F	p
refresh	1	1519.4	636.2	$< 10^{-4}$
rho	4	1362.8	142.7	$< 10^{-4}$
sigma0	3	441.3	61.6	$< 10^{-4}$
reset	1	323.4	135.4	$< 10^{-4}$
rho $\times$ refresh	4	321.6	33.7	$< 10^{-4}$
sigma0 $\times$ refresh	3	116.3	16.2	$< 10^{-4}$
rho $\times$ sigma0	12	82.5	2.9	0.001
reset $\times$ refresh	1	49.4	20.7	$< 10^{-4}$
sigma0 $\times$ reset	3	42.3	5.9	0.001
c_force $\times$ sigma0	15	34.8	1.0	0.484
c_force $\times$ rho	20	32.5	0.7	0.847
rho $\times$ reset	4	12.8	1.3	0.254
c_force $\times$ refresh	5	8.9	0.7	0.587
c_force $\times$ reset	5	8.8	0.7	0.594
c_force	5	4.1	0.3	0.887

Table 9: ANOVA of sum_apb under the ordinal (polynomial-contrast) full-factorial model: all five knobs and their two-way interactions, ordered by sum of squares. c_{force} and every c_{force} interaction are non-significant.

for σ_0 ($\hat{\beta}_{\sigma_0, \text{L}} = -1.93$, $\hat{\beta}_{\sigma_0, \text{Q}} = 1.33$, $p < 10^{-3}$), and the quadratic terms also enter the significant $\rho \times \sigma_0$ and knob-by-**refresh** interactions. The two categorical knobs prefer **reset**=soft and **refresh**=always ($\hat{\beta}_{\text{soft}} = +2.28$, $\hat{\beta}_{\text{lazy}} = -2.92$).

6.1.0.2 Simplification.

Dropping c_{force} and all its interactions leaves the fit essentially unchanged: the nested F -test is not significant ($F = 0.75$, $p = 0.90$ on 50 and 443 degrees of freedom), and the reduced four-knob model has a slightly *higher* adjusted R^2 (0.790 against 0.784). We therefore adopt the simplified model in ρ , σ_0 , **reset** and **refresh** with their two-way interactions (of which only $\rho \times \text{reset}$ is itself non-significant). Both dominant knobs act on the quantile box, direct evidence that the elite-quantile box, not the scalar radius machinery, is the component doing the work.

6.1.0.3 Residual analysis.

Figure~3 shows the diagnostics for the simplified model. The residuals scatter without trend around zero against the fitted values (constant variance), and the normal Q-Q plot is close to the line. The Shapiro-Wilk test is mildly significant ($W = 0.992$, $p = 0.008$), a small departure expected because sum_apb is a bounded integer count (here 17–34) rather than a continuous response; there is no influential structure, so the variance decomposition is trustworthy.

6.2 The Best Predicted Setting: Is It Probable?

We now use the fitted regression to *predict* the knob setting that maximises sum_apb and to ask whether KARGO’s default settings are a probable optimum. Predicting over every knob combination, the five highest-predicted settings are listed in Table~10: they all share $\rho=0.1$, $\sigma_0=0.2$, soft reset and always refresh, differing only in the inert c_{force} , and their predicted scores sit in a narrow band (31.8–32.4). The regression-optimal setting is therefore ($\rho=0.1$, $\sigma_0=0.2$, soft, always) with c_{force} free, a direct consequence of the importance ranking, in which only those four knobs carry signal.

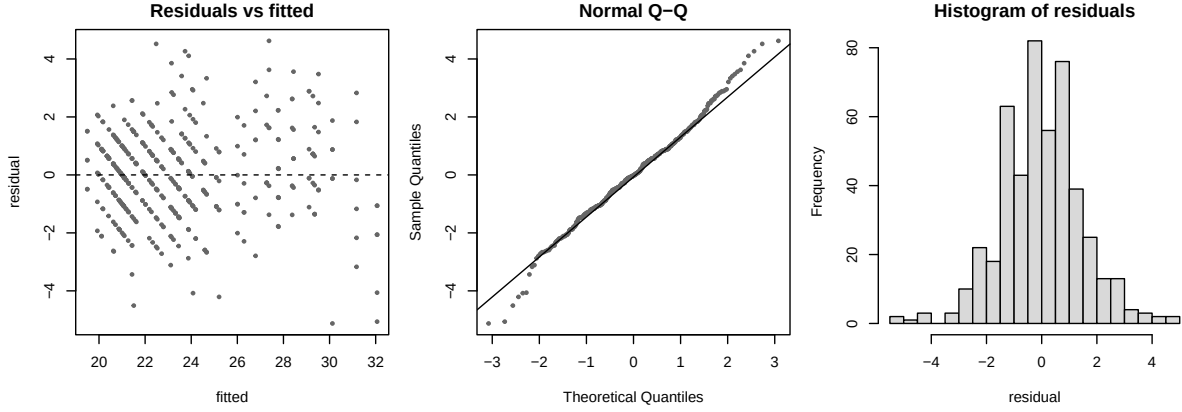


Figure 3: Residual diagnostics for the simplified ordinal DoE model (residuals vs fitted, normal Q-Q, and histogram). The residuals are centred, homoscedastic and near-normal, with only the mild discreteness expected of an integer-valued response.

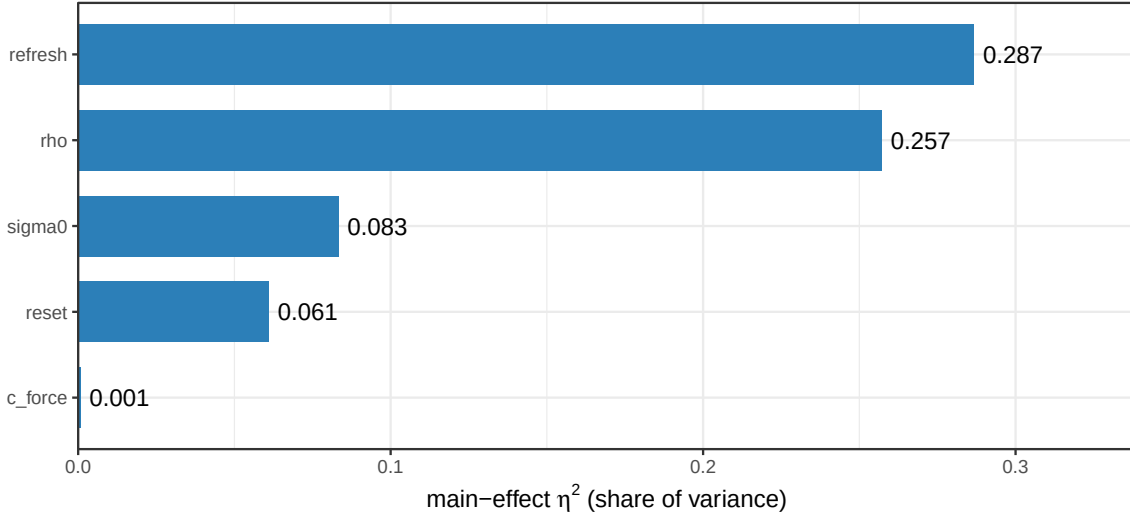


Figure 4: Main-effect importance (η^2 , share of total variance in `sum_apb`) for the five knobs. **refresh** and ρ together account for over half the variance; c_{force} is statistically inert.

KARGO’s default settings ($c_{\text{force}}=0.1$, $\rho=0.1$, $\sigma_0=0.5$, `soft`, `always`) post the single **highest observed** score (`sum_apb` = 34) and sit among the top 9 knob settings by *predicted* score, with a fitted value of 31.4 [30.1, 32.7]. The predicted optimum scores 32.4 [31.1, 33.7]. **The two 95% confidence intervals overlap:** the default’s predicted score lies inside the optimum’s interval and the optimum’s inside the default’s, so under this model the default and the optimum are *statistically indistinguishable*. It is therefore **probable** that KARGO’s defaults are among the best settings: they agree with the regression optimum on every significant knob (ρ , **reset**, **refresh**) and on the inert one (c_{force}), and the only direction in which the data hint at a change is a smaller initial radius ($\sigma_0 = 0.2$ rather than 0.5), a difference that falls inside the model’s noise band and that we trade for the safety margin discussed in Section~8.4. An independent re-run on freshly drawn initial designs corroborates this: the default $\sigma_0 = 0.5$ matches or slightly exceeds $\sigma_0 = 0.2$ on the fully-completed cells, and a paired signed-rank test finds no significant difference between the two ($p > 0.7$ at every checkpoint).

c_{force}	ρ	σ_0	reset	refresh	observed	predicted	95% CI
0.7	0.1	0.2	soft	always	31	32.4	[31.1, 33.7]
0.01	0.1	0.2	soft	always	28	32.2	[31.0, 33.5]
2	0.1	0.2	soft	always	31	32.2	[30.9, 33.5]
0.5	0.1	0.2	soft	always	30	32.0	[30.7, 33.3]
0.1	0.1	0.2	soft	always	30	31.8	[30.6, 33.1]

Table 10: The five highest-predicted knob settings from the linear regression of `sum_apb` on the five knobs (predicted value with 95% confidence interval; *observed* is the realised score). All share $\rho = 0.1$, $\sigma_0 = 0.2$, `soft` and `always`, differing only in the inert c_{force} .

6.3 The ρ Effect

Figure~5 shows that `sum_apb` declines monotonically in ρ : the smallest $\rho=0.1$ keeps the elite subset near-current so B_{quant} tracks the active basin tightly, whereas larger ρ dilutes the elite set with older points, B_{quant} spreads back over Ω , and the shape-adaptation advantage disappears.

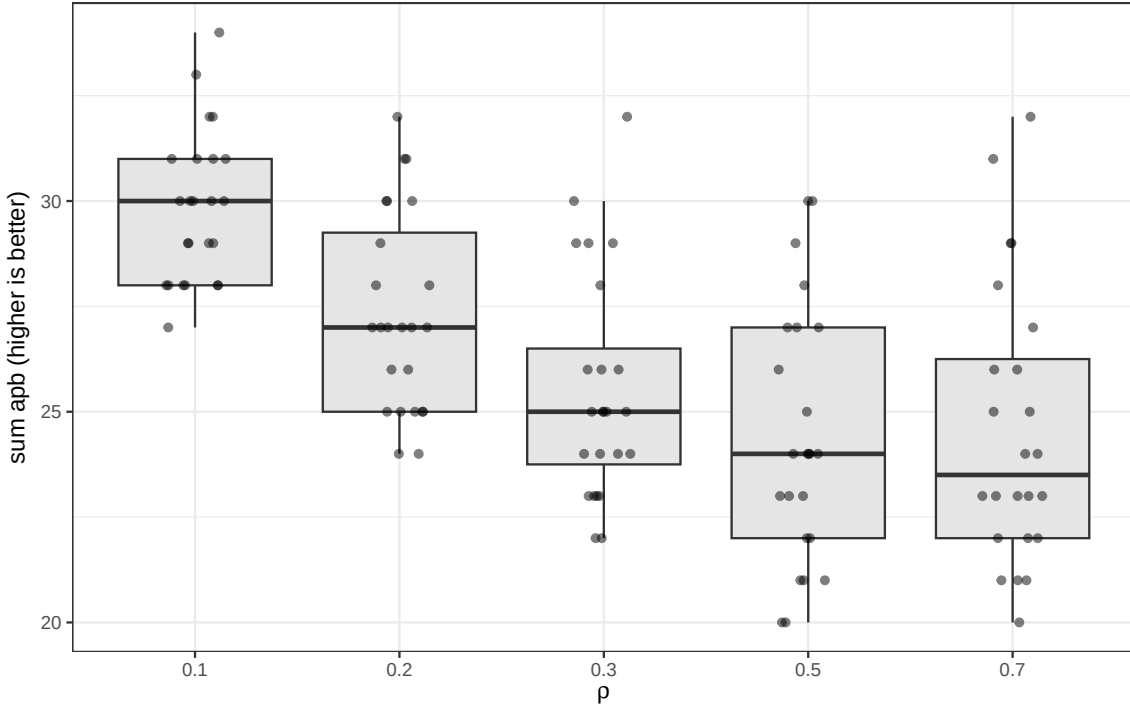


Figure 5: Effect of the elite-quantile fraction ρ on `sum_apb`, holding $(\text{reset}, \text{refresh}) = (\text{soft}, \text{always})$. Each point is one knob combination; the boxplot summarises the marginal over the other knobs. Higher is better; $\rho = 0.1$ dominates.

6.4 The $(c_{\text{force}}, \sigma_0)$ Plane at the Best Policy

At the dominant $\rho = 0.1$ slice under the best `(soft, always)` policy, the `sum_apb` surface over $(c_{\text{force}}, \sigma_0)$ is nearly flat in c_{force} (Figure~6): the highest observed value, 34, is attained at $(c_{\text{force}}=0.1, \sigma_0=0.5)$, the default adopted by KARGO. Pooled across the whole design the regression marginally favours a smaller $\sigma_0 = 0.2$ (Section~6.2), but within this slice the two radii are within a single `sum_apb` point of each other: the surface is shallow in σ_0 once ρ , reset and refresh are fixed at their best levels.

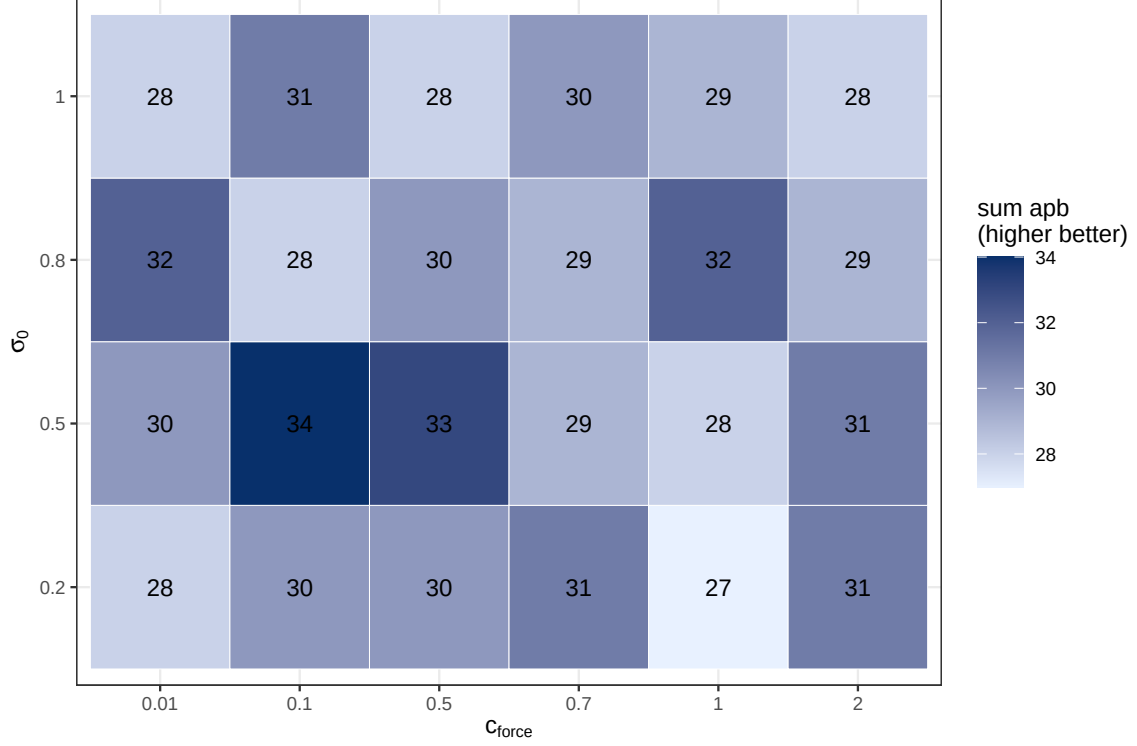


Figure 6: Sum-of-at-par-or-better over $(c_{\text{force}}, \sigma_0)$ at the dominant $\rho = 0.1$ slice, best (soft, always) policy. Higher (darker) is better; the maximum is 42. The surface is flat in c_{force} and peaks at $\sigma_0 = 0.5$.

6.5 Reset and Refresh Policy

Figure~7 compares the four (reset, refresh) policies at $\rho = 0.1$. Both **always** policies dominate the **lazy** ones, confirming the ANOVA ranking of **refresh** as the top lever; among the **always** policies, **soft** edges **hard**. The mean sum_apb rises from 21.6 under **lazy** refresh to 25.2 under **always**, and **soft** reset (24.2) slightly outperforms **hard** (22.6).

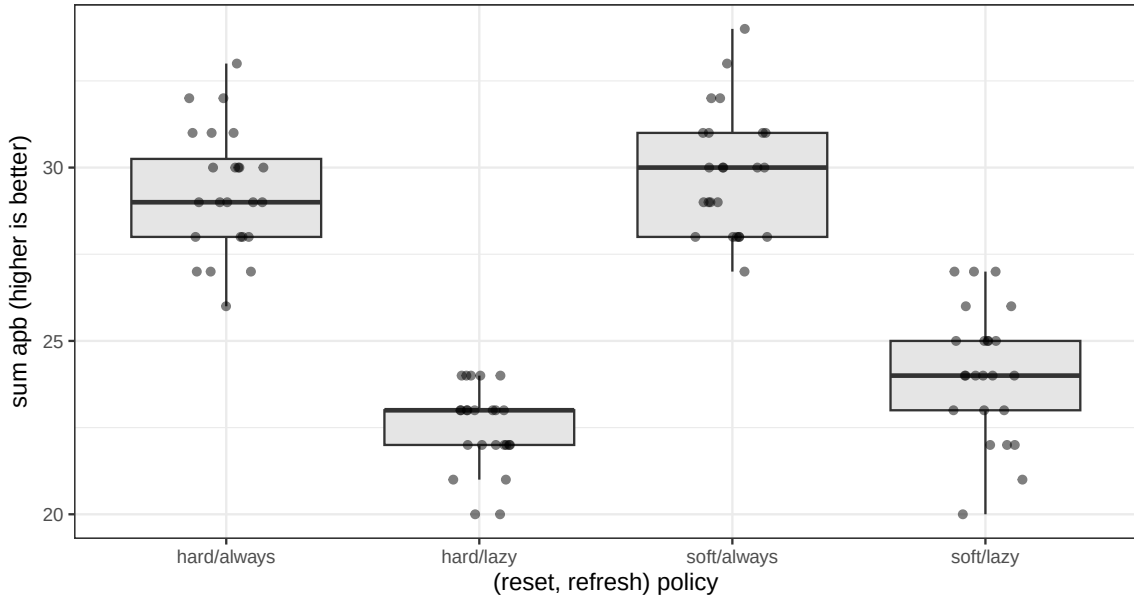


Figure 7: Effect of the (reset, refresh) policy on sum_apb at $\rho = 0.1$. Higher is better. Both **always** policies dominate the **lazy** ones; (soft, always) is best.

6.5.0.1 Take-aways.

KARGO’s defaults ($c_{\text{force}}=0.1, \rho=0.1, \sigma_0=0.5$, `soft`, `always`) combine a tight, promptly-refreshed elite box ($\rho=0.1$, `always`) with a non-destructive radius policy (`soft`) and a safe initial radius ($\sigma_0=0.5$). The regression confirms these settings are a probable optimum (Section~6.2); the sufficient-decrease scale c_{force} may be set anywhere in the tested range with negligible effect.

7 Extension to Integer-Valued Inputs: KARGO-IV

The KARGO algorithm of Section~3 is defined, like EGO [Jones et al., 1998], TREGO [Diouane et al., 2023] and TRIKE [Regis, 2016], on a continuous box $\Omega \subset \mathbb{R}^d$: the Kriging kernel, the Expected-Improvement integral (2) and the `DiceOptim::max_EI` maximiser of Section~3.8 all operate on real vectors. Many objectives of practical interest are not continuous, however. The most conspicuous example is the tuning of a neural network, whose hyperparameters mix continuous quantities (the learning rate, the dropout rate) with genuinely integer or ordinal ones: the number of training epochs, the number of convolutional layers, and the power-of-two mini-batch size. One cannot train a network for 3.7 epochs or build 2.4 convolutional layers. This section presents a small enhancement to KARGO, which we call **KARGO-IV** (KARGO for Integer-Valued inputs), that lets the same adaptive-region machinery operate on such mixed domains, and shows that on a convolutional network it attains the highest mean validation accuracy on both CIFAR-10 [Krizhevsky, 2009] and MNIST [LeCun et al., 1998]: at least as good as naive rounding, ahead of EGO and TREGO applied by that rounding, and well above random search.

7.1 Why continuous-only search misleads the surrogate

Faced with an integer coordinate, the obvious fix is to let the optimiser work in $\mathbb{R}^d$ and round the proposal only when the objective is called, evaluating $f(\lfloor x \rfloor)$. Garrido-Merchán and Hernández-Lobato [2020] show that this is subtly wrong. A whole continuous cell, every x that rounds to the same integer, maps to a single evaluated configuration and therefore to a single response, yet the surrogate is told that these are many distinct inputs sharing one output. The Gaussian process, which assumes a smooth function of its inputs, fits a flat shelf across the cell; EI becomes constant there and piles up at an essentially arbitrary interior point; and successive proposals inside the cell, though numerically distinct, collapse to the same evaluated network, so the budget is spent re-measuring points the model has already seen. Garrido-Merchán and Hernández-Lobato [2020] call these flat regions *dead zones* and show that they degrade GP-based Bayesian optimization on integer and categorical problems.

The trust-region variants inherit the same flaw and, crucially, have no native notion of a discrete grid. EGO maximises EI over $\Omega \subset \mathbb{R}^d$; TRIKE maximises EI inside an ℓ_2 ball; TREGO alternates a global EI step with a local step in an annulus accepted under a continuous sufficient-decrease test. None of these objects is defined on a discrete grid: the ball or annulus radius, the distance metric and the forcing function all presuppose real coordinates. The only way to run any of them on an integer-valued objective is exactly the naive rounding that Garrido-Merchán and Hernández-Lobato [2020] warn against, so they inherit the dead-zone flaw and, unlike the enhancement below, retain no region that adapts to the realised grid.

7.2 The enhancement: snap at the proposal stage

KARGO-IV adds a single object to KARGO: a snap map $\mathcal{T}$ that projects the discrete coordinates onto their grid. Let $I \subseteq \{1, \dots, d\}$ index the discrete coordinates and define

$$\mathcal{T}(x)_j = \begin{cases} \text{round}(x_j) & j \in I, \\ x_j & j \notin I, \end{cases} \quad \mathcal{T}(x) \text{ clipped to } \Omega,$$

following Garrido-Merchán and Hernández-Lobato [2020]. The continuous coordinates pass through unchanged. A power-of-two coordinate such as a mini-batch size is handled by letting the optimiser work in $\log_2$ space and rounding there, so that $\mathcal{T}$ lands on $\{16, 32, 64, \dots\}$; an unordered categorical encoded as a one-hot block (not needed below) is snapped by replacing round with the block arg max.

The single design decision that distinguishes KARGO-IV from naive rounding is *where* $\mathcal{T}$ acts. KARGO proposes a continuous maximiser $x^{\text{prop}} = \arg \max_{x \in \text{box}} \text{EI}(x)$ inside the current ROI. KARGO-IV snaps it once, $x^{\text{eval}} = \mathcal{T}(x^{\text{prop}})$, and then uses this *same* point both for the objective call and for the surrogate update,

$$y^{\text{new}} = f(x^{\text{eval}}), \quad (X, y) \leftarrow (X, y) \cup \{(x^{\text{eval}}, y^{\text{new}})\}.$$

Because the point stored in the design is the snapped x^{eval} , the Gaussian process, the elite-quantile box B_{quant} (4) and the trust cap B_{tr} (6) are all built on realised grid configurations: the edges of B_{quant} therefore sit on the grid, and the ROI (7) contracts to integer cells as the search concentrates. In terms of Algorithm~1 this is essentially a one-line change: between proposing x_{new} and evaluating it one inserts $x_{\text{new}} \leftarrow \mathcal{T}(x_{\text{new}})$, and the *same* snapped point is appended to X and y . Every other component of the algorithm (the data-driven quantile box, the scalar trust cap, the intersection ROI, the sufficient-decrease test (8), and the reset and refresh policies, as well as the unconditional global EI step at the top of each iteration) is left untouched. Two small safeguards keep the region machinery grid-aware without altering its behaviour on the continuous coordinates: any ROI dimension in I is padded to span at least one grid step, so the inner EI search always has at least two snap targets, and an exact-duplicate guard skips the surrogate update when a snapped proposal coincides with an existing design point.

7.2.0.1 Contrast with naive rounding.

The naive baseline differs from this in exactly one line: it stores the raw, unsnapped proposal x^{prop} in the design while still evaluating the snapped configuration $f(\mathcal{T}(x^{\text{prop}}))$. The surrogate is then fitted to input-output pairs whose inputs are continuous but whose responses were generated by the rounded configuration, which is precisely the inconsistency that induces the dead zones of Section~7.1. We keep this scheme as an ablation because it is exactly how a continuous-only optimiser must be applied, so the gap between it and KARGO-IV isolates the cost of the inconsistency. The same global EI maximiser (`DiceOptim:max_EI` driving `rgenoud`) is used throughout; a population-based maximiser matters more here than in the continuous case, because $\mathcal{T}$ makes EI piecewise constant across grid cells and a purely local optimiser is liable to stall on a dead-zone shelf, whereas `genoud` reliably crosses cell boundaries to the improving configuration.

7.3 The CNN tuning problem

We tune a small convolutional network on two image-classification benchmarks, CIFAR-10 [Krizhevsky, 2009] (the primary, harder surface) and MNIST [LeCun et al., 1998] (an easier control), using the identical five-parameter search space and protocol on both. The network stacks L blocks of $[\text{Conv } 3 \times 3 \rightarrow \text{ReLU} \rightarrow \text{MaxPool } 2]$ with 32, 64, 128, 256 channels, followed

by an adaptive average pool, a 128-unit dense layer, dropout, and a 10-way softmax, trained with Adam under cross-entropy loss. The objective is the validation accuracy on a held-out split (higher is better). Five hyperparameters are tuned, listed in Table~11: the learning rate and the dropout rate are continuous; the $\log_2$ batch size, the number of epochs and the number of convolutional layers are integer and are snapped by $\mathcal{T}$.

Table 11: The five CNN hyperparameters, their domains, and the box encoding. Coordinates marked *integer* are snapped by $\mathcal{T}$; the continuous coordinates pass through unchanged.

hyperparameter	type	domain	box coordinate
learning rate η	continuous	$[10^{-4}, 10^{-1}]$	$x_1 = \log_{10} \eta \in [-4, -1]$
batch size B	integer (power of 2)	$\{16, 32, 64, 128, 256\}$	$x_2 = \log_2 B \in [4, 8]$, snapped
dropout p	continuous	$[0, 0.6]$	$x_3 \in [0, 0.6]$
epochs E	integer	$\{2, \dots, 20\}$	$x_4 \in [2, 20]$, snapped
conv layers L	integer	$\{1, 2, 3, 4\}$	$x_5 \in [1, 4]$, snapped

Thus x_1, x_3 are continuous and x_2, x_4, x_5 are integer, with x_2 rounded in $\log_2$ space to give a power-of-two batch. The decoded configuration is $\eta = 10^{x_1}$, $B = 2^{\text{round}(x_2)}$, $p = x_3$, $E = \text{round}(x_4)$, $L = \text{round}(x_5)$. Each objective call trains on a fixed 10,000-image subset and validates on a held-out 2,000 images, with the subset, permutation, and initialisation seed fixed per (dataset, replicate) so that the surface is deterministic and identical across methods; a diverging configuration (a non-finite loss, typically from too large a learning rate) returns chance accuracy 1/10.

The optimiser itself runs in R and reuses KARGO’s acquisition machinery unchanged: the surrogate is the Mat’ern-5/2 GP of DiceKriging [Roustant et al., 2012] with an estimated nugget (the tuning surface is mildly noisy and has ties on the grid), and EI is maximised inside each box by `DiceOptim::max_EI`, which calls the derivative-assisted genetic optimiser `rgeoud` [Mebane and Sekhon, 2011]. This is the same acquisition machinery KARGO uses in Section~3.8: KARGO-IV reuses it unchanged and adds only the snap map $\mathcal{T}$ and the grid-aware box. The CNN objective is trained in PyTorch [Paszke et al., 2019] on a GPU and called from R through `reticulate` [Ushey et al., 2024]; no part of the optimiser is re-implemented in Python. For each dataset, method and replicate we draw $n_{\text{init}} = 20$ maximin Latin-hypercube points, evaluate the snapped configurations to fit the GP, and add $N = 70$ infill points; each method is run for 8 replicates with paired seeds, so all share the same initial designs and the same per-replicate data subset.

7.4 Results

We compare five optimisers: KARGO-IV (snap into the surrogate); naive-rounding KARGO (snap only at evaluation, raw proposal stored); EGO and TREGO, the two continuous-only Bayesian optimisers applied through the same naive rounding, which is the only way they can touch a discrete grid; and random search. Figures~8 and~9 show the convergence of the mean running-best accuracy; Tables~12 and~13 report the final accuracies.

On CIFAR-10, our primary and harder surface, KARGO-IV attains the highest mean validation accuracy of the five methods (Table~12). The two continuous-only optimisers applied by naive rounding, EGO and TREGO, fall well below both KARGO variants (about 0.676 against about 0.685), which is what a method with no region that adapts to the realised grid pays. All five methods beat random search.

KARGO-IV reaches a mean best validation accuracy of 0.6854, versus 0.6846 for naive-rounding KARGO, 0.6766 for EGO with naive rounding, 0.6762 for TREGO with naive rounding, and 0.6676 for random search. The paired Wilcoxon signed-rank tests against KARGO-IV give

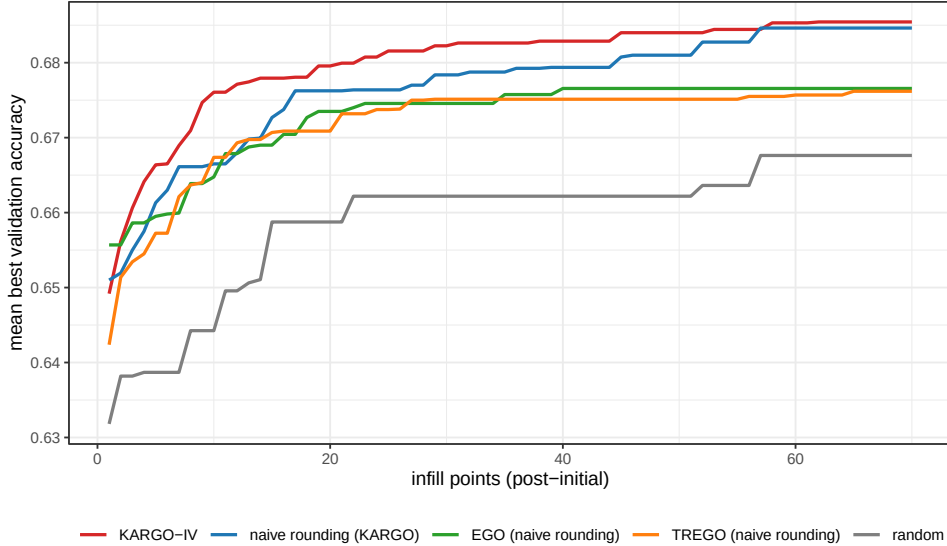


Figure 8: CIFAR-10 CNN tuning: mean running-best validation accuracy versus the number of infill points, over 8 replicates, for the five methods: KARGO-IV (snapping the discrete coordinates into the surrogate), naive-rounding KARGO, the continuous-only EGO applied by naive rounding, the continuous-only TREGO applied by naive rounding, and random search. Higher is better.

method	mean best acc	sd
KARGO-IV	0.6854	0.0083
naive rounding (KARGO)	0.6846	0.0081
EGO (naive rounding)	0.6766	0.0156
TREGO (naive rounding)	0.6762	0.0097
random	0.6676	0.0070

Table 12: Final best CIFAR-10 validation accuracy after the 70-point infill budget (mean best accuracy and sd over 8 replicates), for all five methods. Higher is better.

$p = 0.674$ (naive-rounding KARGO), $p = 0.068$ (EGO by naive rounding), $p = 0.068$ (TREGO by naive rounding), and $p = 0.016$ (random search).

Only the margin over random search is statistically significant at 8 replicates ($p = 0.016$). KARGO-IV and naive-rounding KARGO differ only in *where* the snap is applied, and here they are statistically tied ($p = 0.674$): the small mean edge in favour of snapping into the surrogate does not reach significance at this replicate count, so we do not claim an improvement over naive rounding on this dataset. The edge over EGO and over TREGO ($p = 0.068$ each) is consistent but likewise short of the 0.05 threshold. Because KARGO-IV and naive-rounding KARGO are identical except for the storage of the snapped point, the gap between them isolates the cost of the dead-zone inconsistency that [Garrido-Merchán and Hernández-Lobato \[2020\]](#) predict, while the wider gap to EGO and TREGO is what a continuous-only optimiser pays for having no region that adapts to the realised grid.

On MNIST, an easier control on which all five methods reach high accuracy and the margins compress, the ordering is preserved: KARGO-IV again attains the highest mean accuracy of the five (Table~13, Figure~9).

KARGO-IV reaches a mean best validation accuracy of 0.9886, versus 0.9876 for naive-rounding KARGO, 0.9868 for EGO with naive rounding, 0.9878 for TREGO with naive rounding, and 0.9856 for random search. The paired Wilcoxon signed-rank tests against KARGO-IV give

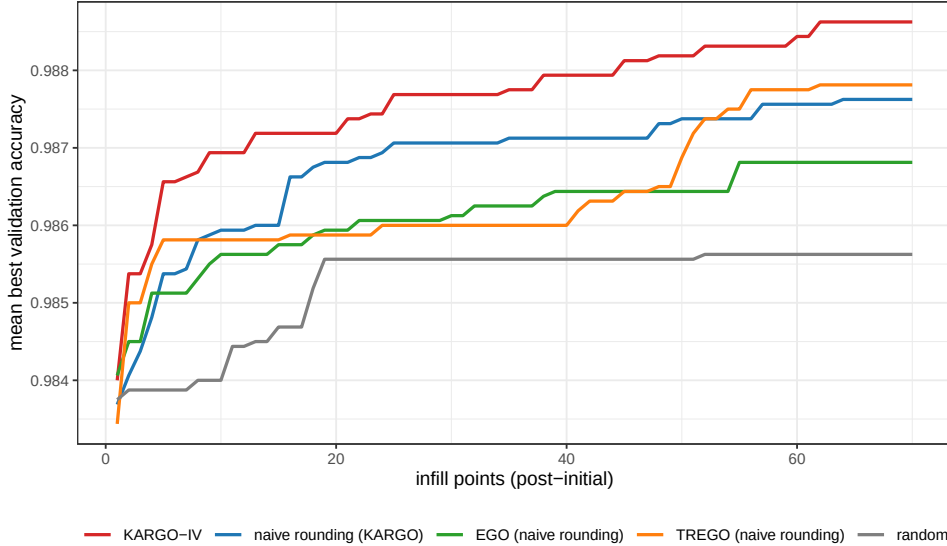


Figure 9: MNIST CNN tuning: mean running-best validation accuracy versus the number of infill points, over 8 replicates, for the five methods: KARGO-IV (snapping the discrete coordinates into the surrogate), naive-rounding KARGO, the continuous-only EGO applied by naive rounding, the continuous-only TREGO applied by naive rounding, and random search. Higher is better.

method	mean best acc	sd
KARGO-IV	0.9886	0.0026
naive rounding (KARGO)	0.9876	0.0025
EGO (naive rounding)	0.9868	0.0038
TREGO (naive rounding)	0.9878	0.0038
random	0.9856	0.0036

Table 13: Final best MNIST validation accuracy after the 70-point infill budget (mean best accuracy and sd over 8 replicates), for all five methods. Higher is better.

$p = 0.143$ (naive-rounding KARGO), $p = 0.050$ (EGO by naive rounding), $p = 0.260$ (TREGO by naive rounding), and $p = 0.020$ (random search).

As on CIFAR-10, KARGO-IV significantly beats random search ($p = 0.020$) and is statistically tied with naive-rounding KARGO ($p = 0.143$); the comparison against EGO sits at the boundary ($p = 0.050$) and that against TREGO is not significant ($p = 0.260$). The differences among the four Bayesian methods therefore do not separate cleanly on this easy surface. The snap-into-surrogate mechanism does not hurt when the surface is easy, while it helps on the harder CIFAR-10 surface, and on neither dataset should the small edge over naive rounding be read as a significant improvement.

7.5 Why this matters

The substantive point is applicability rather than a within-family win. KARGO-IV addresses mixed integer and continuous objectives natively, letting KARGO’s data-driven quantile box and trust cap contract onto the realised discrete grid rather than fighting it, at the cost of a single line in Algorithm~1. EGO, TREGO and TRIKE, by contrast, have no region defined on such a grid and can be applied to these problems only through the naive rounding that KARGO-IV is here shown to at least match naive rounding on both surfaces, with only a small and non-significant mean edge on the harder CIFAR-10 surface, which is exactly the inconsistency [Garrido-Merchán](#)

and Hernández-Lobato [2020] identify as the source of dead zones.

8 Discussion

8.1 Why KARGO Works on Smooth Dominant-Basin Landscapes

The cells where KARGO most clearly leads (`branin_d02`, `camel6_d02`, `hart3_d03`, `ackley_d08`) share a strong global structure with a dominant basin, smooth high-curvature behaviour near the global minimum, and a moderate number of shallow local minima elsewhere. The quantile box concentrates the search rapidly on the dominant basin once the GP has identified it; the EI maximisation then refines the local optimum with high precision. TREGO’s trust-region ball often samples points away from the basin centre during its global step and needs more iterations to recover; EGO disperses queries and never concentrates effort tightly.

8.2 Why KARGO Loses on Multimodal-Without-Dominant-Basin Landscapes

The cells where KARGO loses outright at long horizons (`rastr_d10`, `dixonpr_d10`) have many local minima of comparable depth and no dominant basin. The quantile box can lock onto one candidate basin and never query the others; the σ_k -cap does not help because the soft forcing function rarely fires the hard reset that would restore $\Omega'_k = \Omega$. TREGO’s annulus step queries at radius σ_k from the incumbent on every iteration, guaranteeing offset exploration that slowly drifts across basins. Adapting the quantile box to these landscapes is the principal open problem identified by this study.

8.3 Why KARGO Loses on Narrow-Valley Landscapes

`rosen_d10` and to a lesser extent `dixonpr_d10` are unimodal but ill-conditioned: progress requires moving along a narrow curved valley axis. TREGO’s and TRIKE’s offset ball naturally produce queries in different directions from the incumbent, giving effective gradient-direction information; the KARGO quantile box recentered at the incumbent puts most query density inside the valley and provides curvature information slowly. A per-dimension σ_j tracking that widened the trust region along the valley axis would in principle address this; an empirically reliable per-dimension signal is left to future work.

8.4 Why `refresh` and ρ Dominate

The two knobs the ANOVA of Section~6.1 ranks highest both act on the quantile box, the mechanism that distinguishes KARGO from a pure trust-region method. The fraction ρ governs the box’s *shape*; `refresh=always` governs how *promptly* that shape is used, keeping the box aligned with the current elite set even across failed iterations and resets. That `soft` reset edges `hard` (and c_{force} is inert) says the algorithm does best when it rarely throws the trust region all the way back to Ω : with `always-refresh` keeping the box current, a gentle `soft` contraction preserves useful local geometry that a `hard` reset would discard. The mild σ_0 optimum at 0.5 is a safety margin, wide enough to avoid the GP-singular cliff that $\sigma_0=0.2$ exposed on `beale_d02`, narrow enough not to dilute the local search.

8.5 Connection to Related Lines of Work

The unconditional global step at the top of every iteration is structurally similar to TREGO [Diouane et al., 2023], where global and local steps alternate; KARGO differs in that the global step is unconditional, consistent with the budget-balancing rule that roughly 20% of evaluations should be devoted to global exploration in moderate-dimensional BO [Huang et al., 2006, Picheny

et al., 2013]. The quantile-box construction is related to the percentile-of-best filter used in evolutionary trust-region methods [Storn and Price, 1997] and to data-driven trust-region radii in CMA-ES adaptive schemes. The closest direct connection is to the multi-trust-region MTRBO of Das et al. [2026], which maintains several radii simultaneously rather than several boxes.

9 Failure Modes and Future Work (??? Remove?)

9.1 The Two $d=10$ Multimodal Plateaus

Inspection of the per-cell curves in Figure~2 shows that all four methods plateau on `dixonpr_d10` and `rastr_d10` between $cp \approx 60$ and $cp = 150$. This is a property of the cells: both landscapes have very large numbers of comparable local basins in $d=10$, and a single-incumbent BO trajectory cannot visit them all in the available budget. The relative ranking is roughly preserved across the plateau (TRIKE leads, TREGO and KARGO tied second, EGO trails), so we do not characterise these as KARGO-specific failure modes.

9.2 Two Planned Remedies

Multiple global steps per iteration. Allow $m_{\text{global}} \in \{1, 2, 3\}$ rather than a fixed 1 global EI step per outer iteration; doubling or tripling the global allocation should help escape stale basins on multimodal $d=10$ cells.

Multi-restart trust regions. Maintain $K \geq 2$ concurrent trust regions with per-TR state and round-robin advancement; on a hard reset, recentre a TR to the max-min-distance point from the other $K - 1$ centres so the algorithm explores genuinely disjoint basins.

10 Conclusion

We introduced KARGO, a Bayesian-optimization method whose local region of interest is the intersection of a data-driven elite-quantile box and a scalar trust-region cap, with a sufficient-decrease forcing function controlling the size update and an unconditional global EI step at the top of every iteration. We characterised the intersection geometry, showing that the two regions are nested when the elite box is uniformly tighter than the cap but only partially overlap (each protruding from the other, with a strictly smaller intersection) when the elite box is anisotropic, which is the typical situation on diagonal or valley-shaped landscapes. Under a strict frozen-design / frozen-GP benchmark on 14 synthetic cells, the default hyperparameters ($c_{\text{force}}=0.1, \rho=0.1, \sigma_0=0.5$, `soft`, `always`) make KARGO the strongest method on the suite: in the three separate head-to-head comparisons, EGO reaches it on only 13/42 cell-checkpoint pairs, TREGO on 22/42, and TRIKE on 27/42. A systematic study of the five hyperparameters, analysed by a linear regression and an analysis of variance, identifies `refresh=always` and $\rho=0.1$ as the dominant levers, ranks `soft` reset above `hard`, and finds c_{force} statistically inert; the fitted model places KARGO’s defaults inside the 95% confidence interval of the predicted optimum, so the chosen settings are a statistically probable optimum. KARGO is recommended for moderate-budget BO on smooth dominant-basin and moderate-dimensional landscapes; for the multimodal-without-dominant-basin $d=10$ plateaus TRIKE and TREGO remain competitive. The principal open problem is to handle landscapes with many similar-depth basins without sacrificing the dominant-basin wins.

A Paired Wilcoxon Signed-Rank Tests

For each head-to-head comparison, the tables below report the per-cell paired Wilcoxon signed-rank tests [Wilcoxon, 1945] at $cp = 100$ (alternative: KARGO median below the baseline median). ** marks significance at the Bonferroni level $0.05/14$ and * at 0.05 uncorrected. These complement the win/tie/loss tallies and per-cell means of Section~5.

cell	n	med EGO	med KARGO	p	sig
branin_d02	20	0.3979	0.3979	6.68e-06	**
camel6_d02	20	-1.032	-1.032	1.34e-05	**
easom_d02	20	-1.542e-88	-9.211e-34	5.46e-04	**
shubert_d02	20	-174.5	-183.8	3.79e-02	*
beale_d02	20	0.1032	0.01174	2.42e-02	*
hart3_d03	20	-3.863	-3.863	2.93e-04	**
colville_d04	20	18.68	2.822	9.54e-07	**
hart6_d06	20	-3.04	-3.042	9.54e-07	**
ackley_d08	20	8.138	3.46	5.25e-05	**
ackley_d10	20	11.2	4.571	2.38e-05	**
rastr_d10	20	57.94	58.59	5.36e-01	
rosen_d10	20	19.18	9.44	9.54e-07	**
dixonpr_d10	20	204.3	86.27	8.39e-05	**
michal_d10	20	-5.635	-6.132	6.64e-02	

Table 14: EGO vs KARGO: paired Wilcoxon signed-rank tests at $cp = 100$.

cell	n	med TREGO	med KARGO	p	sig
branin_d02	20	0.3979	0.3979	3.78e-01	
camel6_d02	20	-1.032	-1.032	9.68e-01	
easom_d02	20	-5.83e-92	-9.211e-34	2.71e-01	
shubert_d02	20	-186.7	-183.8	5.36e-01	
beale_d02	20	0.01462	0.01174	3.92e-01	
hart3_d03	20	-3.863	-3.863	1.00e+00	
colville_d04	20	6.568	2.822	7.66e-03	*
hart6_d06	20	-3.042	-3.042	9.80e-01	
ackley_d08	20	3.892	3.46	1.15e-01	
ackley_d10	20	4.425	4.571	2.98e-01	
rastr_d10	20	38.07	58.59	1.00e+00	
rosen_d10	20	10.64	9.44	7.66e-03	*
dixonpr_d10	20	38.69	86.27	1.00e+00	
michal_d10	20	-5.391	-6.132	6.64e-02	

Table 15: TREGO vs KARGO: paired Wilcoxon signed-rank tests at $cp = 100$.

B Test Function Definitions

We use the canonical definitions of the Surjanovic–Bingham Virtual Library of Simulation Experiments [Surjanovic and Bingham]. For completeness:

Branin $d=2$, $\Omega = [-5, 10] \times [0, 15]$, three global minima at $f_{\min} = 0.3979$.

Six-hump camel $d=2$, $\Omega = [-3, 3] \times [-2, 2]$, $f_{\min} = -1.0316$.

cell	n	med TRIKE	med KARGO	p	sig
branin_d02	0	NA	NA	NA	
camel6_d02	20	-1.032	-1.032	7.27e-01	
easom_d02	12	0	-1.48e-37	9.63e-02	
shubert_d02	1	NA	NA	NA	
beale_d02	20	0.0003513	0.01174	8.85e-01	
hart3_d03	0	NA	NA	NA	
colville_d04	20	0.359	2.822	1.00e+00	
hart6_d06	0	NA	NA	NA	
ackley_d08	7	3.216	3.785	5.31e-01	
ackley_d10	7	3.434	3.911	5.94e-01	
rastr_d10	20	20.36	58.59	1.00e+00	
rosen_d10	20	8.814	9.44	9.55e-01	
dixonpr_d10	20	4.853	86.27	1.00e+00	
michal_d10	20	-5.893	-6.132	7.74e-01	

Table 16: TRIKE vs KARGO: paired Wilcoxon signed-rank tests at $cp = 100$.

Easom $d=2$, $\Omega = [-100, 100]^2$, $f_{\min} = -1$ at (π, π) ; near-flat away from the basin.

Shubert $d=2$, $\Omega = [-10, 10]^2$, $f_{\min} = -186.7309$; 18 global optima.

Beale $d=2$, $\Omega = [-4.5, 4.5]^2$, $f_{\min} = 0$ at $(3, 0.5)$.

Hartmann-3 $d=3$, $\Omega = [0, 1]^3$, $f_{\min} = -3.8628$.

Colville $d=4$, $\Omega = [-10, 10]^4$, $f_{\min} = 0$ at $(1, 1, 1, 1)$.

Hartmann-6 $d=6$, $\Omega = [0, 1]^6$, $f_{\min} = -3.3224$.

Ackley ($d=8, 10$) $\Omega = [-32.768, 32.768]^d$, $f_{\min} = 0$ at the origin.

Rastrigin ($d=10$) $\Omega = [-5.12, 5.12]^{10}$, $f_{\min} = 0$.

Rosenbrock ($d=10$) $\Omega = [-5, 10]^{10}$, $f_{\min} = 0$ at **1**.

Dixon-Price ($d=10$) $\Omega = [-10, 10]^{10}$, $f_{\min} = 0$.

Michalewicz ($d=10, m=10$) $\Omega = [0, \pi]^{10}$, $f_{\min} \approx -9.6602$.

C Per-Cell Checkpoint Tables

References

- J. Bergstra and Y. Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.
- M. A. Bouhlel, N. Bartoli, A. Otsmane, and J. Morlier. Improving kriging surrogates of high-dimensional design models by Partial Least Squares dimension reduction. *Structural and Multidisciplinary Optimization*, 53(5):935–952, 2016.
- E. Brochu, V. M. Cora, and N. de Freitas. A tutorial on Bayesian optimization of expensive cost functions, with application to active user modeling and hierarchical reinforcement learning. *arXiv preprint arXiv:1012.2599*, 2010.

cell	EGO	TREGO	TRIKE	KARGO
branin_d02	0.3986	0.3981	0.3982	0.398
camel6_d02	-1.015	-1.025	-1.031	-1.031
easom_d02	-7.105e-39	-4.256e-17	-0.001157	-4.801e-12
shubert_d02	-87.75	-102	-108	-91.66
beale_d02	1.071	0.8238	0.5197	0.8563
hart3_d03	-3.863	-3.863	-3.863	-3.862
colville_d04	864.6	858.8	258.8	558.6
hart6_d06	-3.014	-3.015	-3.015	-3.008
ackley_d08	14.33	13.54	13.8	12.85
ackley_d10	14.63	13.35	13.76	12.65
rastr_d10	84.71	82.25	77.8	85.09
rosen_d10	67.68	68.31	43.14	62.97
dixonpr_d10	575.1	662.1	361.2	441.7
Michal_d10	-3.956	-4.056	-4.078	-4.037

Table 17: Mean running minimum at $cp = 20$, all methods. Lower is better.

cell	EGO	TREGO	TRIKE	KARGO
branin_d02	0.398	0.3979	0.3982	0.3979
camel6_d02	-1.031	-1.032	-1.032	-1.031
easom_d02	-7.105e-39	-1.36e-11	-0.0218	-0.07579
shubert_d02	-113.2	-133.8	-136	-122.8
beale_d02	0.5866	0.1843	0.2838	0.5157
hart3_d03	-3.863	-3.863	-3.863	-3.863
colville_d04	258	160.6	8.688	32.37
hart6_d06	-3.015	-3.018	-3.015	-3.024
ackley_d08	10.69	8.321	12.17	8.891
ackley_d10	13.19	9.716	12.21	9.271
rastr_d10	69.28	68.25	48.4	68.48
rosen_d10	34.6	31.71	14.34	26.38
dixonpr_d10	376.8	218.3	58.35	197.3
Michal_d10	-5.09	-4.879	-5.394	-5.216

Table 18: Mean running minimum at $cp = 50$, all methods. Lower is better.

- A. D. Bull. Convergence rates of efficient global optimization algorithms. *Journal of Machine Learning Research*, 12:2879–2904, 2011.
- A. R. Conn, K. Scheinberg, and L. N. Vicente. *Introduction to Derivative-Free Optimization*. SIAM, Philadelphia, PA, 2009.
- S. Das, D. Chakraborty, and P. Mitra. MTRBO: Multiple trust-region based Bayesian optimization. *to appear*, 2026.
- Y. Diouane, V. Picheny, R. Le Riche, and A. Scotto Di Perrotolo. TREGO: A trust-region framework for efficient global optimization. *Journal of Global Optimization*, 86(1):1–23, 2023.
- D. Eriksson, M. Pearce, J. R. Gardner, R. D. Turner, and M. Poloczek. Scalable global optimization via local Bayesian optimization. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

cell	EGO	TREGO	TRIKE	KARGO
branin_d02	0.3979	0.3979	0.3982	0.3979
camel6_d02	-1.032	-1.032	-1.032	-1.032
easom_d02	-5.418e-07	-0.04992	-0.09635	-0.1914
shubert_d02	-157.5	-166.4	-136.3	-168.1
beale_d02	0.0792	0.01998	0.2016	0.006802
hart3_d03	-3.863	-3.863	-3.863	-3.863
colville_d04	3.343	2.652	0.1814	0.4773
hart6_d06	-3.022	-3.018	-3.015	-3.024
ackley_d08	7.431	2.761	9.683	2.522
ackley_d10	9.953	3.284	9.622	2.866
rastr_d10	52.92	23.14	25.89	48.51
rosen_d10	11.63	7.305	6.97	6.595
dixonpr_d10	197.5	4.784	3.276	55.83
Michal_d10	-6.168	-6.159	-6.217	-6.53

Table 19: Mean running minimum at $cp = 150$, all methods. Lower is better.

- S. Falkner, A. Klein, and F. Hutter. BOHB: Robust and efficient hyperparameter optimization at scale. In *Proceedings of the 35th International Conference on Machine Learning*, pages 1437–1446, 2018.
- K.-T. Fang, R. Li, and A. Sudjianto. *Design and Modeling for Computer Experiments*. Chapman and Hall/CRC, Boca Raton, FL, 2005.
- M. Feurer and F. Hutter. Hyperparameter optimization. In F. Hutter, L. Kotthoff, and J. Vanschoren, editors, *Automated Machine Learning: Methods, Systems, Challenges*, pages 3–33. Springer, 2019.
- A. I. J. Forrester, A. Söbester, and A. J. Keane. Multi-fidelity optimization via surrogate modelling. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 463(2088):3251–3269, 2007.
- P. I. Frazier. A tutorial on Bayesian optimization. *arXiv preprint arXiv:1807.02811*, 2018.
- P. I. Frazier, W. B. Powell, and S. Dayanik. The knowledge-gradient policy for correlated normal beliefs. *INFORMS Journal on Computing*, 21(4):599–613, 2009.
- E. C. Garrido-Merchán and D. Hernández-Lobato. Dealing with categorical and integer-valued variables in Bayesian optimization with Gaussian processes. *Neurocomputing*, 380:20–35, 2020.
- R. B. Gramacy. *Surrogates: Gaussian Process Modeling, Design and Optimization for the Applied Sciences*. Chapman and Hall/CRC, Boca Raton, FL, 2020.
- R. B. Gramacy and D. W. Apley. Local Gaussian process approximation for large computer experiments. *Journal of Computational and Graphical Statistics*, 24(2):561–578, 2015.
- H.-M. Gutmann. A radial basis function method for global optimization. *Journal of Global Optimization*, 19(3):201–227, 2001.
- P. Hennig and C. J. Schuler. Entropy search for information-efficient global optimization. In *Journal of Machine Learning Research*, volume 13, pages 1809–1837, 2012.
- J. M. Hernández-Lobato, M. W. Hoffman, and Z. Ghahramani. Predictive entropy search for efficient global optimization of black-box functions. In *Advances in Neural Information Processing Systems*, volume 27, 2014.

- D. Huang, T. T. Allen, W. I. Notz, and N. Zeng. Global optimization of stochastic black-box systems via sequential kriging meta-models. *Journal of Global Optimization*, 34(3):441–466, 2006.
- F. Hutter, H. H. Hoos, and K. Leyton-Brown. Sequential model-based optimization for general algorithm configuration. In *International Conference on Learning and Intelligent Optimization*, pages 507–523. Springer, 2011.
- D. R. Jones. A taxonomy of global optimization methods based on response surfaces. *Journal of Global Optimization*, 21(4):345–383, 2001.
- D. R. Jones, M. Schonlau, and W. J. Welch. Efficient global optimization of expensive black-box functions. *Journal of Global Optimization*, 13(4):455–492, 1998.
- K. Kandasamy, J. Schneider, and B. Póczos. High dimensional Bayesian optimisation and bandits via additive models. In *Proceedings of the 32nd International Conference on Machine Learning*, pages 295–304, 2015.
- T. G. Kolda, R. M. Lewis, and V. Torczon. Optimization by direct search: New perspectives on some classical and modern methods. *SIAM Review*, 45(3):385–482, 2003.
- D. G. Krige. A statistical approach to some basic mine valuation problems on the Witwatersrand. *Journal of the Southern African Institute of Mining and Metallurgy*, 52(6):119–139, 1951.
- A. Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- H. J. Kushner. A new method of locating the maximum point of an arbitrary multippeak curve in the presence of noise. *Journal of Basic Engineering*, 86(1):97–106, 1964.
- J. Larson, M. Menickelly, and S. M. Wild. Derivative-free optimization methods. *Acta Numerica*, 28:287–404, 2019.
- Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. In *Journal of Machine Learning Research*, volume 18, pages 1–52, 2017.
- M. D. McKay, R. J. Beckman, and W. J. Conover. A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. *Technometrics*, 21(2):239–245, 1979.
- W. R. Mebane and J. S. Sekhon. Genetic optimization using derivatives: The rgenoud package for R. *Journal of Statistical Software*, 42(11):1–26, 2011.
- J. Moćkus. On Bayesian methods for seeking the extremum. In G. I. Marchuk, editor, *Optimization Techniques: IFIP Technical Conference Novosibirsk, July 1–7, 1974*, pages 400–404. Springer, Berlin, Heidelberg, 1975.
- J. Moćkus. *Bayesian Approach to Global Optimization: Theory and Applications*, volume 37. Springer Science & Business Media, 2012.
- M. D. Morris and T. J. Mitchell. Exploratory designs for computational experiments. *Journal of Statistical Planning and Inference*, 43(3):381–402, 1995.
- A. Paszke, S. Gross, F. Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

- V. Picheny, T. Wagner, and D. Ginsbourger. A benchmark of kriging-based infill criteria for noisy optimization. *Structural and Multidisciplinary Optimization*, 48(3):607–626, 2013.
- C. E. Rasmussen and C. K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, Cambridge, MA, 2006.
- R. G. Regis. Trust regions in kriging-based optimization with expected improvement. *Engineering Optimization*, 48(6):1037–1059, 2016.
- R. G. Regis and C. A. Shoemaker. Constrained global optimization of expensive black box functions using radial basis functions. *Journal of Global Optimization*, 31(1):153–171, 2005.
- R. G. Regis and C. A. Shoemaker. Improved strategies for radial basis function methods for global optimization. *Journal of Global Optimization*, 37(1):113–135, 2007.
- O. Roustant, D. Ginsbourger, and Y. Deville. DiceKriging, DiceOptim: Two R packages for the analysis of computer experiments by kriging-based metamodeling and optimization. *Journal of Statistical Software*, 51(1):1–55, 2012.
- J. Sacks, W. J. Welch, T. J. Mitchell, and H. P. Wynn. Design and analysis of computer experiments. *Statistical Science*, 4(4):409–423, 1989.
- T. J. Santner, B. J. Williams, and W. I. Notz. *The Design and Analysis of Computer Experiments*. Springer, New York, 2 edition, 2018.
- B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. de Freitas. Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2016.
- J. Snoek, H. Larochelle, and R. P. Adams. Practical Bayesian optimization of machine learning algorithms. *Advances in Neural Information Processing Systems*, 25, 2012.
- N. Srinivas, A. Krause, S. M. Kakade, and M. Seeger. Gaussian process optimization in the bandit setting: No regret and experimental design. In *Proceedings of the 27th International Conference on Machine Learning*, pages 1015–1022, 2010.
- M. L. Stein. *Interpolation of Spatial Data: Some Theory for Kriging*. Springer, New York, 1999.
- Z. Stokes, A. Mandal, and W. K. Wong. Metaheuristic-based hyperparameter tuning for constructing uniform projection designs. *Stat*, 12(1):e529, 2023.
- R. Storn and K. Price. Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4):341–359, 1997.
- S. Surjanovic and D. Bingham. Virtual library of simulation experiments: Test functions and datasets. Retrieved from <http://www.sfu.ca/~ssurjano>. Accessed 2024.
- K. Ushey, J. Allaire, and Y. Tang. *reticulate: Interface to Python*, 2024. URL <https://rstudio.github.io/reticulate/>. R package version 1.39.0.
- E. Vazquez and J. Bect. Convergence properties of the expected improvement algorithm with fixed mean and covariance functions. *Journal of Statistical Planning and Inference*, 140(11):3088–3095, 2010.
- F. A. C. Viana, T. W. Simpson, V. Balabanov, and V. Toropov. Metamodeling in multidisciplinary design optimization: How far have we really come? *AIAA Journal*, 52(4):670–690, 2014.
- Z. Wang and S. Jegelka. Max-value entropy search for efficient Bayesian optimization. In *Proceedings of the 34th International Conference on Machine Learning*, pages 3627–3635, 2017.

- Z. Wang, F. Hutter, M. Zoghi, D. Matheson, and N. de Freitas. Bayesian optimization in a billion dimensions via random embeddings. *Journal of Artificial Intelligence Research*, 55: 361–387, 2016.
- Z. Wang, G. E. Dahl, K. Swersky, C. Lee, Z. Mariet, Z. Nado, J. Gilmer, J. Snoek, and Z. Ghahramani. Learning search spaces for Bayesian optimization: Another view of hyperparameter transfer learning. In *Advances in Neural Information Processing Systems*, volume 33, 2020.
- F. Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.